COMPLETE LEARNING COMPANION

Angular Mastery

Grounded in the LifeTrack Clinical Trial Management Platform

9 Topics · 55 Interview Q&As · Full Project Code

Angular v21 · Standalone Components · ASP.NET Core Microservices Backend

Prepared for Hari Hara Sudhan — Cognizant ACE 2026

A complete transcript of every concept, code example, and explanation

Table of Contents

TOPIC 1 OF 11

Introduction to Angular & Setting Up the Environment

Definition

Angular is a **platform and framework** for building single-page client applications using HTML and TypeScript. It is developed and maintained by Google and follows a component-based architecture where the entire UI is built from self-contained, reusable pieces called components.

Explanation

Angular is not just a library (like React) — it is a full-fledged, opinionated framework. It comes batteries-included with:

- A **component system** for structuring the UI
- **Two-way data binding** to sync view and model
- A **dependency injection** system for services
- Built-in **routing** for navigation
- A powerful **CLI** to scaffold, build, and test apps
- **RxJS** integration for reactive async programming

Angular apps are written in **TypeScript** (a typed superset of JavaScript), and compiled down to JavaScript for the browser. Every Angular app has one root module and one root component, and everything else branches from there.

Sub-topics

1. Overview of Angular
2. Installing Angular CLI
3. Creating a new Angular project
4. Angular project structure and files
5. Running and building the app

Diagram covered: Angular ecosystem overview

The browser (client) layer holds Components (HTML + TS templates), Services (business logic, DI injectable), Router (URL → Component), and RxJS (async streams). These connect through HttpClient (GET/POST/PUT/DELETE) to the Backend API (ASP.NET Core + Ocelot) — the LifeTrack microservices on ports 5001–5008. At build time, the Angular CLI toolchain (`ng new` , `ng serve` , `ng build` , `ng generate`) compiles and bundles everything.

Diagram covered: Angular project folder structure

`lifetrack-frontend/` → `src/` → `app/` (your components, services, modules), `app.component.ts` (root component class), `app.component.html` (root template), `app.routes.ts` (route definitions). Plus `environments/` (API URLs for dev/prod), `index.html` (shell HTML containing `<app-root>`), `main.ts` (bootstraps the app — entry point), `angular.json` (CLI workspace config), `package.json` (npm dependencies), and `tsconfig.json` (TypeScript compiler options).

Sub-topic: What `ng serve` does under the hood

When you type `ng serve` , the Angular CLI kicks off a pipeline with several distinct phases. Each phase is a real, separate process.

Phase 1 — Read angular.json and resolve config

The CLI reads your `angular.json` workspace file and resolves which project to serve, which builder to use (`@angular-devkit/build-angular:dev-server`), and merges any flags you passed (like `--port` or `--configuration`). Key options resolved: port 4200, host localhost, ssl false. Your CORS fix lives here — port 4200 locked in angular.json so the ASP.NET CORS whitelist always matches.

Phase 2 — TypeScript compilation (tsc)

The TypeScript compiler reads `tsconfig.app.json` and type-checks every `.ts` file in your `src/`. It does NOT emit JS files to disk — the AST is handed directly to esbuild in memory. Type errors are reported here and abort the build. If you mistype a property on `AdverseEventDto`, tsc catches it here before the browser sees anything.

Phase 3 — Angular compilation (ngtsc, AOT)

The Angular template compiler reads every `@Component` decorator, compiles the HTML template into TypeScript factory functions (`ɵcmp`), resolves all directives/pipes used in the template, and produces the final compiled component definitions. This is what turns `<app-protocol-list>` into actual DOM instructions. Template errors caught here: unknown pipe, missing `*ngIf` import, wrong property binding type. Each standalone component's `imports[]` array is resolved here — no NgModule needed.

Phase 4 — Bundling (esbuild)

Since Angular 17, the default bundler is esbuild — much faster than the old Webpack. It tree-shakes dead code, combines all your modules, and produces a small set of JS and CSS files (`main.js`, `polyfills.js`, `styles.css`) held in memory (not written to disk during `ng serve`). Source maps are generated in memory so DevTools can show original `.ts` source. **This is the key difference from `ng build`, which writes these chunks to `/dist/` on disk.**

Phase 5 — Dev server (Vite)

Angular's dev server (backed by Vite since Angular 17) serves the in-memory bundle over HTTP at `http://localhost:4200`. It injects a WebSocket client into the page for Hot Module Replacement (HMR) at `ws://localhost:4200/__hmr`. A `proxy.conf.json` can forward `/api/*` to `localhost:5000` (Ocelot).

Phase 6 — Browser bootstrap

The browser fetches `index.html`, which has a `<script>` tag pointing to `main.js`. Angular's `bootstrapApplication()` mounts `AppComponent` into `<app-root>`. The DI container is built (all providers from appConfig — HttpClient, Router, services — instantiated). The router reads the URL and activates the matching component. Change detection starts (Zone.js or signals begins watching for state changes).

Phase 7 — File watch loop

After the initial build, `ng serve` keeps a file system watcher (chokidar) running, watching `src/**/*.{ts,html,scss}`. When you save a file, it re-runs only the affected phases (usually just phases 3–4 for a component change). Only the changed file plus its dependents are recompiled — not the whole app. The HMR websocket pushes the new module to the browser without a full reload. Changes to `angular.json`, `tsconfig.json`, or `app.config.ts` trigger a full reload.

> **THE THREE THINGS THAT MAKE NG SERVE SPECIAL**
>
> **1. Everything stays in memory** — unlike `ng build`, nothing is written to disk. That's why `/dist/` doesn't appear.
>
> **2. The Angular template compiler runs before the browser sees anything (AOT)** — your HTML templates are converted to JS factory functions before bundling. This is why you get template errors at serve time, not runtime.
>
> **3. HMR only re-runs phases 3 and 4** — when you save a component, the Angular compiler recompiles only that component, esbuild re-bundles the affected chunk, and Vite pushes just that module patch over a WebSocket.

> **WHY YOUR CORS ISSUE HAPPENED**
>
> When `ng serve` starts the Vite dev server, your app's HTTP calls to `http://localhost:5000` (Ocelot) originate from `http://localhost:4200`. The browser enforces CORS — it checks whether the backend lists `http://localhost:4200` in its `Access-Control-Allow-Origin` header. When the port wasn't locked, it could vary (4200, 4201, 4202...), so the whitelist never matched reliably. Locking the port fixed it because ASP.NET's `.WithOrigins("http://localhost:4200")` would always match.

`ng serve` vs `ng build --watch`

	ng serve	ng build --watch
Output location	In memory	/dist/ on disk
Dev server	Yes (Vite on :4200)	No — separate server needed
HMR	Yes	No (full re-bundle on change)
Use case	Day-to-day development	Testing the production build locally

Sub-topic: What `angular.json` controls

`angular.json` is the **workspace configuration file** — the single source of truth that tells the Angular CLI how to build, serve, test, and lint every project in your workspace. It controls: which project is the default, the build builder and its options (output path, assets, styles, scripts, budgets), the serve configuration (port, host, proxy), the test configuration (Karma), file replacements per environment (swapping `environment.ts` for `environment.prod.ts` during production builds), and optimization settings (AOT, tree-shaking, minification).

Code Examples

1. Installing the Angular CLI

terminal

```
# Install Angular CLI globally
npm install -g @angular/cli

# Verify installation
ng version
```

2. Creating your LifeTrack frontend project

terminal

```
# Create new Angular 21 standalone project
ng new lifetrack-frontend --standalone --routing --style=scss

# Move into the project
cd lifetrack-frontend

# Start the dev server — app runs on localhost:4200
ng serve --open
```

The `--standalone` flag is key — your LifeTrack app uses the **standalone component pattern** (no `NgModule` files), the modern Angular 17+ approach.

3. main.ts — the entry point

src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

// This is what Angular runs first.
// It mounts AppComponent into the <app-root> tag in index.html
bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

4. app.config.ts — configuring providers (standalone pattern)

src/app/app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),         // routing
    provideHttpClient(),           // for calling your Ocelot gateway
  ]
};
```

5. LifeTrack environment.ts — pointing at your gateway

src/environments/environment.ts (dev)

```
export const environment = {
  production: false,
  apiBaseUrl: 'http://localhost:5000'  // Ocelot gateway port
};
```

src/environments/environment.prod.ts (prod)

```
export const environment = {
  production: true,
  apiBaseUrl: 'https://your-prod-gateway.com'
};
```

6. ng build — producing a production bundle

terminal

```
# Compiles, minifies, tree-shakes
ng build --configuration production

# Output goes to /dist/lifetrack-frontend/
# These static files can be served from IIS, Nginx, or Azure Static Apps
```

ng CLI Commands Cheat Sheet

Command	Purpose
`ng new <name>`	Scaffold a new workspace. Flags: --standalone, --routing, --style=scss
`ng serve`	Compile + start dev server with HMR. Default port 4200
`ng generate component`	Generate a component (ts+html+scss+spec). Alias: ng g c
`ng generate service`	Generate an injectable service. Alias: ng g s
`ng build --configuration production`	Compile, minify, tree-shake for deployment. Output: /dist/
`ng test`	Run unit tests using Karma + Jasmine
`ng generate guard`	Generate a router guard. Used in LifeTrack for role-based route protection

LIFETRACK CONNECTION

Your LifeTrack project uses `ng serve` on port 4200; the CORS fix whitelisted `http://localhost:4200` in your ASP.NET Core services; `environment.ts` points `apiBaseUrl` to `http://localhost:5000` (Ocelot gateway); all components follow the `--standalone` pattern; services like `ProtocolService`, `AeService` are generated with `ng g s`.

KNOWLEDGE CUTOFF NOTE ON THE PROJECT'S ACTUAL STRUCTURE

The real project is `Lifetrack.UI` and actually uses the NgModule pattern (`standalone: false`) with `AppModule` , `AuthModule` , and a lazy-loaded `DashboardModule` — not the pure standalone bootstrap shown in these introductory examples. The standalone examples above illustrate the modern Angular 17+ approach; the project's true module-based structure is detailed in Topics 5 and 6.

TOPIC 2 OF 11

Angular Components

Definition

A component is the fundamental building block of every Angular application. It controls a patch of the screen called a **view** and is made up of three things that always travel together: a TypeScript class (the logic), an HTML template (the structure), and optional CSS (the styles).

Explanation

Think of a component the way you'd think of a LEGO brick — self-contained, reusable, and designed to snap together with other bricks. In LifeTrack, your entire UI is a tree of these bricks: `AppComponent` at the root, then feature components like `CtmAdverseEventsPageComponent`, `CtmDashboardComponent`, each rendered inside one another. Every Angular component is declared with the `@Component` decorator, which tells Angular "this class is a component, not just a plain TypeScript class."

Diagram covered: anatomy of @Component

The `@Component({ })` decorator sits on your TS class. Config/DI properties: `selector` (HTML tag name), `standalone: true` (no NgModule needed), `imports: []` (other components, pipes, directives), `providers: []` (component-scoped services). View/rendering properties: `templateUrl` (path to .html), `styleUrls: []` (scoped CSS), `changeDetection` (Default or OnPush), `encapsulation` (CSS scope strategy). Below sits `export class AeListComponent` with properties, methods, and lifecycle hooks.

Sub-topics covered

1. Creating components
2. Data binding — property, event, and two-way
3. Lifecycle hooks (`ngOnInit` , `ngOnChanges` , etc.)
4. Parent-child communication (`@Input` and `@Output`)

1. Creating a Component

terminal

```
# Generate a standalone AE list component inside the ctm folder
ng generate component ctm/ae-list --standalone
```

This creates four files: `ae-list.component.ts` (class + decorator), `ae-list.component.html` (template), `ae-list.component.scss` (scoped styles), `ae-list.component.spec.ts` (unit test file).

src/app/ctm/ae-list/ae-list.component.ts

```ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { AeService } from '../../services/ae.service';
import { AdverseEvent } from '../../models/adverse-event.model';

@Component({
  selector: 'app-ae-list',        // used as <app-ae-list> in templates
  standalone: true,               // no NgModule needed — modern Angular pattern
  imports: [CommonModule],        // gives access to *ngIf, *ngFor, async pipe
  templateUrl: './ae-list.component.html',
  styleUrls: ['./ae-list.component.scss']
})
export class AeListComponent implements OnInit {
  adverseEvents: AdverseEvent[] = [];

  constructor(private aeService: AeService) {}   // DI: service injected here

  ngOnInit(): void {
    this.aeService.getAll().subscribe(data => this.adverseEvents = data);
  }
}
```

2. Data Binding — the four types

Diagram covered: four data binding types with direction

Interpolation (`{{ title }}`) and property binding (`[disabled]="isDisabled"`) flow class → template. Event binding (`(click)="onSubmit()"`) flows template → class. Two-way binding (`[(ngModel)]="searchTerm"`) flows both directions.

ae-list.component.html

```html
<!-- 1. INTERPOLATION — reads a string property, renders as text -->
<h2>{{ pageTitle }}</h2>
<!-- renders: "Adverse Events" -->

<!-- 2. PROPERTY BINDING — binds a class property to an element property -->
<button [disabled]="isLoading">Submit AE</button>
<img [src]="trialLogoUrl" [alt]="trialName" />

<!-- 3. EVENT BINDING — calls a class method when a DOM event fires -->
<button (click)="onApproveAe(ae.id)">Approve</button>
<input (keyup)="onSearch($event)" placeholder="Search AEs..." />

<!-- 4. TWO-WAY BINDING — syncs input value ↔ class property -->
<input [(ngModel)]="searchTerm" placeholder="Filter by keyword" />
```

ae-list.component.ts

```ts
export class AeListComponent {
  pageTitle = 'Adverse Events';
  isLoading = false;
  searchTerm = '';          // two-way bound to the input above

  onApproveAe(id: number) {
    this.isLoading = true;
    this.aeService.approve(id).subscribe(() => this.isLoading = false);
  }

  onSearch(event: KeyboardEvent) {
    this.searchTerm = (event.target as HTMLInputElement).value;
  }
}
```

> **NOTE**
>
> For `[(ngModel)]` to work, you must add `FormsModule` to the component's `imports` array.

3. Lifecycle Hooks

Angular calls these methods automatically at specific moments in a component's life. Implement the corresponding interface to hook in.

Diagram covered: lifecycle hook execution order

`constructor()` → `ngOnChanges()` → `ngOnInit()` → `ngDoCheck()` → `ngAfterContentInit()` → `ngAfterContentChecked()` → `ngAfterViewInit()` → `ngAfterViewChecked()` → (update loop repeats ngOnChanges → ngDoCheck → checked hooks on every state change) → `ngOnDestroy()` fires once when the component is removed.

constructor() — runs once

Called immediately when Angular instantiates the class — before inputs are set, before the template renders. Use ONLY to inject services. Do NOT call HTTP requests here.

```
constructor(private aeService: AeService) {
  // Safe: store the injected service
  // Unsafe: this.aeService.getAll() — too early
}
```

ngOnChanges(changes) — repeats

Fires every time a parent changes an `@Input()` property. The `changes` object shows previous and current value. Runs BEFORE `ngOnInit` on the first cycle.

```
ngOnChanges(changes: SimpleChanges): void {
  if (changes['trialId']) {
    const prev = changes['trialId'].previousValue;
    const curr = changes['trialId'].currentValue;
    // reload data when parent changes the trial
    this.loadAeForTrial(curr);
  }
}
```

ngOnInit() — runs once (the workhorse)

Called once after `@Input()` properties are set. The right place to make HTTP calls, subscribe to services, or set up derived state.

```
ngOnInit(): void {
  // Safe: @Input() values are set, fetch data
  this.aeService.getAll().subscribe(data => {
    this.adverseEvents = data;
  });
}
```

ngDoCheck() — repeats (rarely needed)

Fires on every change detection run — very frequent. Only use for custom dirty-checking Angular's default can't detect. Avoid in LifeTrack.

ngAfterContentInit() — runs once

Fires after Angular projects external content via `<ng-content>`. Used when building reusable container components (cards, modals, panels) that accept slotted content.

ngAfterViewInit() — runs once

Called after Angular renders the component's own template and all child components. The right place to access native DOM elements via `@ViewChild`, or initialize third-party JS libraries that need a real DOM node.

```
@ViewChild('chartCanvas') chartRef!: ElementRef;

ngAfterViewInit(): void {
  // DOM is ready — initialize a chart library
  const ctx = this.chartRef.nativeElement;
  new Chart(ctx, { type: 'bar', data: this.kpiData });
}
```

ngOnDestroy() — runs once (cleanup)

Called just before Angular destroys the component. CRITICAL: unsubscribe from Observables here to prevent memory leaks. Also clear timers, detach event listeners, complete Subjects.

```
private destroy$ = new Subject<void>();

ngOnInit(): void {
  this.aeService.getAll()
    .pipe(takeUntil(this.destroy$))
    .subscribe(data => this.adverseEvents = data);
}

ngOnDestroy(): void {
  this.destroy$.next();
  this.destroy$.complete(); // stops all subscriptions
}
```

What exists at each lifecycle hook — the critical mental model

Hook	services	@Input()	DOM/template	@ViewChild	@ContentChild
constructor()	Yes	No	No	No	No
ngOnChanges()	Yes	Yes	No	No	No
ngOnInit()	Yes	Yes	No	No	No
ngDoCheck()	Yes	Yes	No	No	No
ngAfterContentInit()	Yes	Yes	Yes	No	Yes
ngAfterViewInit()	Yes	Yes	Yes	Yes	Yes
ngOnDestroy()	Yes	Yes	Yes	Yes	Yes

THE MOST COMMON MISTAKE — FORGETTING TO UNSUBSCRIBE

User opens AeListComponent → ngOnInit subscribes. User navigates away → Angular destroys the component. User comes back → a new component subscribes again, but the old subscription is still alive. After 5 navigations, 5 duplicate subscriptions are running, each firing callbacks and updating stale component state — a memory leak plus ghost UI updates. The `takeUntil(this.destroy$)` pattern is the clean fix — one `destroy$` subject kills every subscription at once.

Decision guide — which hook?

I want to...	Use this hook
Inject a service	`constructor`
Make an HTTP call on load	`ngOnInit`
React when parent changes an @Input()	`ngOnChanges`
Access a <canvas> or @ViewChild element	`ngAfterViewInit`
Initialize Chart.js / ag-Grid	`ngAfterViewInit`
Unsubscribe from Observables	`ngOnDestroy`
Accept slotted content (ng-content)	`ngAfterContentInit`
Detect a deep object mutation (last resort)	`ngDoCheck`

4. Parent-Child Communication — @Input and @Output

Diagram covered: @Input / @Output parent-child flow

Parent (`ProtocolViewComponent`) holds `selectedProtocolId = 42` . Its template uses `<app-documents-tab [protocolId]="selectedProtocolId" (documentUploaded)="onDocumentUploaded($event)">` . Child (`DocumentsTabComponent`) declares `@Input() protocolId!: number` and `@Output() documentUploaded = new EventEmitter<Document>()` , calling `this.documentUploaded.emit(doc)` on upload. @Input flows parent → child; @Output flows child → parent.

documents-tab.component.ts (CHILD)

```
import { Component, Input, Output, EventEmitter, OnInit } from '@angular/core';
import { DocumentService } from '../../services/document.service';
import { Document } from '../../models/document.model';

@Component({
  selector: 'app-documents-tab',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './documents-tab.component.html'
})
export class DocumentsTabComponent implements OnInit {

  // Receives the protocol ID from parent — readonly from child's perspective
  @Input() protocolId!: number;

  // Fires an event upward when a new document is uploaded
  @Output() documentUploaded = new EventEmitter<Document>();

  documents: Document[] = [];

  constructor(private docService: DocumentService) {}

  ngOnInit(): void {
    // Uses the @Input() to fetch the right documents
    this.docService.getByProtocol(this.protocolId).subscribe(d => this.documents = d);
  }

  onUploadComplete(newDoc: Document): void {
    // Emit the new document up to the parent
    this.documentUploaded.emit(newDoc);
  }
}
```

protocol-view.component.ts (PARENT)

```ts
export class ProtocolViewComponent {
  selectedProtocolId = 42;

  // Called when the child emits documentUploaded
  onDocumentUploaded(doc: Document): void {
    console.log('New doc uploaded in child:', doc.title);
    this.refreshComplianceStatus();
  }
}
```

protocol-view.component.html (PARENT TEMPLATE)

```html
<app-documents-tab
  [protocolId]="selectedProtocolId"
  (documentUploaded)="onDocumentUploaded($event)">
</app-documents-tab>
```

Component interview reference

Question	Answer
Where do you make HTTP calls?	`ngOnInit()` — inputs are set, DOM not needed yet
Where do you access @ViewChild elements?	`ngAfterViewInit()` — the DOM is ready
Where do you prevent memory leaks?	`ngOnDestroy()` — unsubscribe all Observables
Difference between [] and ()?	`[prop]` = data flows in (property binding); `(event)` = data flows out (event binding)
What is [()] (banana in a box)?	Two-way binding — shorthand for `[ngModel]` + `(ngModelChange)`
When does ngOnChanges run?	Before ngOnInit on first render, then every @Input() change
What does standalone: true mean?	The component declares its own imports — no NgModule wrapper

TOPIC 3 OF 11

Directives & Pipes

Definition

A **directive** is a class that tells Angular to do something to a DOM element — show it, repeat it, style it conditionally. A **pipe** is a transformation function used directly inside a template to format data before displaying it. If components are the "what to display", then directives control the "whether/how to display it" and pipes control "how to format what's displayed".

Diagram covered: three categories

Structural directives (add/remove DOM nodes): `*ngIf`, `*ngFor`, `*ngSwitch`, `@if` / `@for`. **Attribute directives** (change look/behaviour of existing nodes): `ngClass`, `ngStyle`, `ngModel`, custom. **Pipes** (transform display values): `date`, `number`, `async`, `uppercase`, custom pipe.

The missed sub-topic: *ngSwitch

`*ngSwitch` is the Angular equivalent of a JavaScript `switch` statement in the template. You use it when one value can match multiple cases and each shows different HTML — a cleaner alternative to chaining multiple `*ngIf` statements. Your project doesn't use it, but here's where it would fit — the role-based subtitle in your AE page currently uses three separate `*ngIf` statements:

```
<!-- CURRENT code in ctm-adverse-events-page.component.html -->
<p *ngIf="isInvestigator">Report and track adverse events...</p>
<p *ngIf="isRegulatoryOfficer">Review and update status...</p>
<p *ngIf="!isInvestigator && !isRegulatoryOfficer">Monitor and manage...</p>
```

```
<!-- EQUIVALENT using *ngSwitch -->
<div [ngSwitch]="userRole">
  <p *ngSwitchCase="'Investigator'">
    Report and track adverse events for your enrolled patients
  </p>
  <p *ngSwitchCase="'RegulatoryOfficer'">
    Review and update status of adverse events across all protocols
  </p>
  <p *ngSwitchCase="'ClinicalTrialManager'">
    Monitor and manage adverse events across all protocols
  </p>
  <p *ngSwitchDefault>
    View adverse events for your assigned role
  </p>
</div>
```

```
// In the TS class — expose a single string instead of three booleans
get userRole(): string {
  return this.authSvc.currentUser?.role ?? '';
}
```

The key rules: `[ngSwitch]` on the parent, `*ngSwitchCase` on each child option, `*ngSwitchDefault` as the fallback. Only the matching case is added to the DOM — all others are destroyed, just like `*ngIf`.

Part 1 — Structural Directives

Structural directives physically add or remove DOM elements. The `*` prefix is the giveaway — it's shorthand that Angular desugars into an `<ng-template>` internally.

*ngIf — from your actual Adverse Events page

ctm-adverse-events-page.component.html

```
<!-- 1. Show a specific subtitle based on role -->
<p class="page-subtitle" *ngIf="isInvestigator">
  Report and track adverse events for your enrolled patients
</p>
<p class="page-subtitle" *ngIf="isRegulatoryOfficer">
  Review and update status of adverse events across all protocols
</p>
<p class="page-subtitle" *ngIf="!isInvestigator && !isRegulatoryOfficer">
  Monitor and manage adverse events across all protocols
</p>

<!-- 2. Show "+ Report AE" button only for Investigators -->
<button class="btn-primary" *ngIf="isInvestigator" (click)="openReportModal()">
  + Report AE
</button>

<!-- 3. Show error alert only if error AND no modal open -->
<div class="alert alert-error"
     *ngIf="error && !showReviewModal && !showReportModal">
  {{ error }}
</div>

<!-- 4. Show loading spinner while HTTP call is in progress -->
<div class="spinner-container" *ngIf="loading">
  <div class="spinner"></div>
</div>
```

ctm-adverse-events-page.component.ts

```
export class CtmAdverseEventsPageComponent implements OnInit {
  loading = false;          // bound to *ngIf="loading"
  error   = '';             // bound to *ngIf="error && ..."
  isInvestigator      = false;
  isRegulatoryOfficer = false;
  showReviewModal = false;
  showReportModal = false;

  ngOnInit(): void {
    const user = this.authSvc.currentUser;
    this.isInvestigator      = user?.role === 'Investigator';
    this.isRegulatoryOfficer = user?.role === 'RegulatoryOfficer';
    this.loadData();
  }
}
```

***NGIF IS NOT DISPLAY:NONE**

When `isInvestigator = false`, the element is **destroyed and removed from the DOM entirely** — not just hidden with `display:none`. The element occupies no memory at all.

*ngIf with else — the #emptyState template

```
<table class="data-table"
       *ngIf="filteredEvents.length > 0; else emptyState">
  <tbody>
    <tr *ngFor="let ae of filteredEvents; let i = index"> ... </tr>
  </tbody>
</table>

<!-- Inserted only when the *ngIf condition is false -->
<ng-template #emptyState>
  <div class="empty-state">
    <p *ngIf="isInvestigator">No adverse events found. Use "+ Report AE" to log one.</p>
    <p *ngIf="!isInvestigator">No adverse events found for the selected filter.</p>
  </div>
</ng-template>
```

The `else emptyState` tells Angular: "when the condition is false, render the template marked `#emptyState` in its place." The `#` makes it a template reference variable.

*ngFor — rendering the AE table rows

```
<tr *ngFor="let ae of filteredEvents; let i = index">
  <td>{{ i + 1 }}</td>
  <td>{{ patientMap[ae.patientID] || 'Patient #' + ae.patientID }}</td>
  <td>{{ protocolMap[ae.protocolID] || 'Protocol #' + ae.protocolID }}</td>
  <td class="description-cell">{{ ae.description }}</td>
  <td>
    <span class="badge" [ngClass]="severityClass(ae.severity)">
      {{ ae.severity }}
    </span>
  </td>
  <td>{{ ae.reportedDate | date:'mediumDate' }}</td>
  <td *ngIf="!isInvestigator">
    <button (click)="openReviewModal(ae)">Review</button>
  </td>
</tr>
```

`ctm-adverse-events-page.component.ts — applyFilter()`

```
adverseEvents: AdverseEvent[] = [];     // full list from API
filteredEvents: AdverseEvent[] = [];    // subset *ngFor iterates

applyFilter(): void {
  this.filteredEvents = this.adverseEvents.filter(ae => {
    const matchStatus   = !this.selectedStatus   || ae.status   === this.selectedStatus;
    const matchSeverity = !this.selectedSeverity || ae.severity === this.selectedSeverity;
    const matchSearch   = !this.searchTerm.trim() ||
      ae.description?.toLowerCase().includes(this.searchTerm.trim().toLowerCase());
    return matchStatus && matchSeverity && matchSearch;
  });
}
```

The local variables `*ngFor` exposes: `index`, `first`, `last`, `even`, `odd` (e.g. for zebra striping).

Angular's new @if / @for syntax (v17+)

```
<!-- OLD syntax (your project uses this) -->
<div *ngIf="loading">Loading...</div>
<tr *ngFor="let ae of filteredEvents">...</tr>

<!-- NEW syntax (Angular 17+ — built into the template engine) -->
@if (loading) { <div>Loading...</div> }
@for (ae of filteredEvents; track ae.eventID) {
  <tr>...</tr>
} @empty {
  <div class="empty-state">No AEs found.</div>
}
```

Part 2 — Attribute Directives

[ngClass] — your severity and status badges

```
<span class="badge" [ngClass]="severityClass(ae.severity)">{{ ae.severity }}</span>
<span class="badge" [ngClass]="statusClass(ae.status)">{{ ae.status }}</span>
```

`ctm-adverse-events-page.component.ts`

```
severityClass(severity: string): string {
  if (severity === 'Critical' || severity === 'Severe') return 'badge-red';
  if (severity === 'Moderate') return 'badge-amber';
  if (severity === 'Mild')     return 'badge-green';
  return 'badge-slate';
}
statusClass(status: string): string {
  if (status === 'Reported')     return 'badge-red';
  if (status === 'Under Review') return 'badge-amber';
  if (status === 'Resolved')     return 'badge-green';
  return 'badge-slate';
}
```

For a "Critical" AE, Angular renders `<span class="badge badge-red">Critical</span>`. The shorthand `[class.row-unread]="n.status === 'Unread'"` applies a single class when the expression is truthy (used in your notifications page).

[ngStyle] / [attr] / [style] — your reports page gauges

`ctm-reports-page.component.html`

```
<circle cx="40" cy="40" r="36"
  [attr.stroke]="gaugeColor(visitComplianceRate)"
  [attr.stroke-dasharray]="gaugeDash(visitComplianceRate)"
  [attr.stroke-dashoffset]="gaugeOffset"/>

<span class="ar-gauge-pct" [style.color]="gaugeColor(visitComplianceRate)">
  {{ visitComplianceRate }}%
</span>

<div class="ar-donut" [style.background]="enrollmentGradient"> ... </div>

<div class="ar-funnel-bar"
  [style.width.%]="f.width"
  [style.background]="f.color"></div>
```

`ctm-reports-page.component.ts`

```
gaugeColor(pct: number): string {
  if (pct >= 70) return '#22c55e';   // green = healthy
  if (pct >= 40) return '#f59e0b';   // amber = warning
  return '#ef4444';                   // red = critical
}
gaugeDash(pct: number): string {
  const circ = 2 * Math.PI * 36;
  const filled = circ * (Math.min(pct, 100) / 100);
  return `${filled} ${circ - filled}`;
}
```

[(ngModel)] — two-way binding in your search and filter bar

```
<input class="filter-search"
  [(ngModel)]="searchTerm"
  (ngModelChange)="onStatusFilterChange()"
  placeholder="Search by description…">

<select class="filter-select" [(ngModel)]="selectedSeverity" (change)="onStatusFilterChange()">
  <option value="">All Severities</option>
  <option value="Mild">Mild</option>
  <option value="Moderate">Moderate</option>
  <option value="Severe">Severe</option>
  <option value="Critical">Critical</option>
</select>
```

FULL DATA FLOW WHEN A USER TYPES "CARDIAC"

User types → `[(ngModel)]` updates `searchTerm = 'cardiac'` → `(ngModelChange)` calls `onStatusFilterChange()` → which calls `applyFilter()` → filters `adverseEvents` where description includes 'cardiac' → `filteredEvents` = matching AEs only → `*ngFor` re-renders the table with only filtered results.

Part 3 — Pipes

The date pipe

```
// notifications-page.component.html
{{ n.createdDate | date:'MMM d, y · h:mm a' }}
// "2026-05-14T08:30:00Z" → "May 14, 2026 · 8:30 AM"

// ctm-adverse-events-page.component.html
{{ ae.reportedDate | date:'mediumDate' }}
// "2026-05-14T00:00:00" → "May 14, 2026"

// ctm-reports-page.component.html
{{ r.generatedDate | date:'d MMM yyyy' }}     // "14 May 2026"
{{ reviewAE.reportedDate | date:'d MMMM yyyy' }}  // "14 May 2026"
```

Token	Meaning	Example
d	Day of month	14
MMM	Short month	May
MMMM	Full month	May
y or yyyy	Year	2026
h	Hour (12h)	8
mm	Minutes	30
a	AM/PM	AM
'mediumDate'	Named preset	May 14, 2026

The number pipe — from your KPI Reports cards

```
<span class="rc-kpi-val">{{ r.enrollmentRate | number:'1.1-1' }}%</span>
<span class="rc-kpi-val">{{ r.dropoutRate | number:'1.1-1' }}%</span>
<span class="rc-kpi-val">{{ r.visitComplianceRate | number:'1.1-1' }}%</span>
// 'minIntDigits.minFracDigits-maxFracDigits' → 87.333333 becomes "87.3"
```

Chaining pipes

```
{{ protocol.title | uppercase | slice:0:20 }}
```

Part 4 — Custom Directive

Your project has no custom attribute directive yet. Here's a `SeverityHighlightDirective` that fits LifeTrack — adds a glowing border to any AE row when severity is Critical:

`src/app/shared/directives/severity-highlight.directive.ts`

```
import { Directive, Input, OnChanges, ElementRef, Renderer2 } from '@angular/core';

@Directive({
  selector: '[appSeverityHighlight]', // used as <tr appSeverityHighlight="Critical">
  standalone: false
})
export class SeverityHighlightDirective implements OnChanges {

  @Input('appSeverityHighlight') severity: string = '';

  constructor(
    private el: ElementRef,       // direct access to the host DOM element
    private renderer: Renderer2   // Angular's safe DOM API
  ) {}

  ngOnChanges(): void {
    this.renderer.removeClass(this.el.nativeElement, 'highlight-critical');
    this.renderer.removeClass(this.el.nativeElement, 'highlight-severe');
    if (this.severity === 'Critical') {
      this.renderer.addClass(this.el.nativeElement, 'highlight-critical');
    } else if (this.severity === 'Severe') {
      this.renderer.addClass(this.el.nativeElement, 'highlight-severe');
    }
  }
}
```

```
<!-- usage -->
<tr *ngFor="let ae of filteredEvents" [appSeverityHighlight]="ae.severity">
  <td>{{ ae.description }}</td>
</tr>
```

Part 5 — Custom Pipe

src/app/shared/pipes/severity-label.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'severityLabel',   // used as: ae.severity | severityLabel
  standalone: false
})
export class SeverityLabelPipe implements PipeTransform {

  transform(severity: string): string {
    const labels: Record<string, string> = {
      'Mild':     'Mild',
      'Moderate': 'Moderate',
      'Severe':   'Severe ⚠',
      'Critical': 'Critical ⛔'
    };
    return labels[severity] ?? severity;
  }
}
```

Mental model: structural vs attribute vs pipe

```
*ngIf="loading"          → DOM node created OR destroyed
*ngFor="let ae of list"  → N DOM nodes created, one per item
[ngClass]="badgeClass()" → existing DOM node gets class attribute modified
[(ngModel)]="searchTerm" → existing DOM input gets value synced both ways
{{ date | date:'...' }}  → value transformed before display — original unchanged
```

> **THE MOST IMPORTANT RULE**
>
> Pipes never mutate your data. `ae.reportedDate` stays as `"2026-05-14T00:00:00"` in your array — the `date` pipe only transforms it for display.

Directives & pipes interview reference

Question	Answer from LifeTrack
*ngIf vs display:none?	*ngIf removes the element from the DOM entirely. display:none keeps it in memory. Use *ngIf for modals.
What does * mean in *ngIf?	Syntactic sugar. Angular desugars it into `<ng-template [ngIf]="...">`.
How to pass a class conditionally?	`[ngClass]="methodThatReturnsClassName()"` or `[class.x]="boolean"`
What's the else in *ngIf for?	Render a fallback ng-template when the condition is false. Used in your AE page empty state.
Can you chain pipes?	Yes: `{{ value | uppercase | slice:0:10 }}`. Left to right.
What does number:'1.1-1' mean?	Min 1 integer digit, min 1 and max 1 decimal place.

TOPIC 4 OF 11 — PART 1

Angular Forms: Template-Driven Forms

Definition

A template-driven form is an Angular form where the structure, validation rules, and data binding are all defined **directly in the HTML template** using directives like `ngModel`, `ngForm`, and `required`. Angular reads the template and automatically builds the form model behind the scenes — you write less TypeScript but have less programmatic control.

How it works — the mental model

Angular's `FormsModule` watches your HTML. The moment it sees a `<form>` tag, it creates a hidden `NgForm` object. The moment it sees `[(ngModel)]` on an input, it creates a hidden `NgModel` object for that field. These hidden objects track the field's value, validity state, and whether the user has touched it — all without you building anything in the TS class. This is the opposite of Reactive Forms, where you build everything in TypeScript first and then connect it to the template.

Diagram covered: template-driven architecture

The HTML template (`<form #f="ngForm">` , `[(ngModel)]="searchTerm"` , `required minlength="3"` , `(ngSubmit)="onSubmit(f)"`) is read by FormsModule, which builds the Angular form model automatically: NgForm (whole form — valid, submitted, value) and NgModel (each field — valid, touched, dirty, errors). `[(ngModel)]` keeps the TS class property and input value in sync both ways.

Your project's template-driven form — the AE Report Modal

Your project uses template-driven form style in `ctm-adverse-events-page.component.html` for the "Report AE" modal.

Step 1 — The TS class side (the data model)

`ctm-adverse-events-page.component.ts`

```
// reportForm is a PLAIN OBJECT — no FormGroup, no FormBuilder
reportForm = {
  patientID:    '',   // bound to the patient <select>
  protocolID:   '',   // bound to the protocol <select>
  description:  '',   // bound to the <textarea>
  severity:     '',   // bound to the severity <select>
  reportedDate: ''    // bound to the date <input>
};

// In template-driven forms, YOU manage errors yourself
reportFormErrors: Record<string, string> = {};

validateReportForm(): boolean {
  this.reportFormErrors = {};
  if (!this.reportForm.patientID)
    this.reportFormErrors['patientID'] = 'Patient is required.';
  if (!this.reportForm.protocolID)
    this.reportFormErrors['protocolID'] = 'Protocol is required.';
  if (!this.reportForm.severity)
    this.reportFormErrors['severity'] = 'Severity is required.';
  if (!this.reportForm.description || !this.reportForm.description.trim()) {
    this.reportFormErrors['description'] = 'Description is required.';
  } else if (this.reportForm.description.trim().length < 10) {
    this.reportFormErrors['description'] = 'Description must be at least 10 characters.';
  } else if (this.reportForm.description.trim().length > 1000) {
    this.reportFormErrors['description'] = 'Description cannot exceed 1000 characters.';
  }
  if (!this.reportForm.reportedDate) {
    this.reportFormErrors['reportedDate'] = 'Reported date is required.';
  } else if (this.reportForm.reportedDate > this.today) {
    this.reportFormErrors['reportedDate'] = 'Reported date cannot be in the future.';
  }
  return Object.keys(this.reportFormErrors).length === 0;
}
```

Step 2 — The HTML template (where directives do the work)

```
<form (ngSubmit)="submitReport()">

  <div class="form-group">
    <label class="form-label" for="rpPatient">Patient <span class="required">*</span></label>
    <select id="rpPatient"
            [(ngModel)]="reportForm.patientID"
            name="rpPatient"
            (ngModelChange)="onReportPatientChange()"
            [class.input-err]="reportFormErrors['patientID']">
      <option value="">— Select Patient —</option>
      <option *ngFor="let p of enrolledPatients" [value]="p.patientID">
        {{ p.name }}
      </option>
    </select>
    <span class="err-msg" *ngIf="reportFormErrors['patientID']">
      {{ reportFormErrors['patientID'] }}
    </span>
  </div>

  <div class="form-group">
    <label for="rpProtocol">Protocol <span class="required">*</span></label>
    <select id="rpProtocol"
            [(ngModel)]="reportForm.protocolID"
            name="rpProtocol"
            [class.input-err]="reportFormErrors['protocolID']"
            [disabled]="!reportForm.patientID">
      <option value="">
        {{ reportForm.patientID ? '— Select Protocol —' : '— Select a patient first —' }}
      </option>
      <option *ngFor="let p of filteredProtocols" [value]="p.protocolID">
        {{ p.title }}
      </option>
    </select>
  </div>

  <div class="form-group">
    <textarea id="rpDescription"
              [(ngModel)]="reportForm.description"
              name="rpDescription"
              rows="4"
              [class.input-err]="reportFormErrors['description']"></textarea>
  </div>

  <div class="form-group">
    <input type="date" id="rpDate"
           [(ngModel)]="reportForm.reportedDate"
           name="rpDate" [max]="today"
           [class.input-err]="reportFormErrors['reportedDate']" />
  </div>

  <div class="modal-footer">
    <button type="button" (click)="closeReportModal()" [disabled]="reportSubmitting">Cancel</button>
    <button type="submit" [disabled]="reportSubmitting">
      {{ reportSubmitting ? 'Submitting…' : 'Submit AE' }}
    </button>
  </div>
</form>
```

The `name=""` attribute is REQUIRED for template-driven forms — Angular uses it to register the field in the NgForm model.

What template-driven forms give you out of the box

Property	CSS class	Meaning
field.valid / invalid	.ng-valid / .ng-invalid	Whether the field passes all validators
field.touched / untouched	.ng-touched / .ng-untouched	Whether the user has focused and left
field.dirty / pristine	.ng-dirty / .ng-pristine	Whether the value changed from initial
field.errors	{ required: true } or null	Which validators failed
field.value	current string/number	The current value
form.submitted	(on NgForm)	True when the user clicks Submit

Using Angular's built-in validators in template-driven forms

```
<textarea name="description" [(ngModel)]="reportForm.description"
  #descField="ngModel"
  required minlength="10" maxlength="1000"></textarea>

<div *ngIf="descField.touched && descField.invalid">
  <span *ngIf="descField.errors?.['required']">Description is required.</span>
  <span *ngIf="descField.errors?.['minlength']">
    Minimum {{ descField.errors?.['minlength'].requiredLength }} characters required.
  </span>
</div>
```

The `#descField="ngModel"` assigns the NgModel directive instance to a template variable, exposing `.valid`, `.invalid`, `.touched`, `.errors` directly.

Diagram covered: form submission flow

User clicks "Submit AE" → `(ngSubmit)` fires → calls `submitReport()` → `validateReportForm()` manually checks each field → decision "valid?" → if No: show errors via `reportFormErrors`; if Yes: HTTP POST to `/adverse-events` with `reportSubmitting = true` → Success closes modal, Error shows `reportError`.

Template-driven vs Reactive comparison

	Template-driven (AE modal)	Reactive (Login/Register)
Form model built in	HTML template	TypeScript class
Module needed	FormsModule	ReactiveFormsModule
Validation defined via	HTML attributes + manual method	Validators array in fb.group()
Unit testable?	Harder (needs DOM)	Easy (pure TypeScript)
Dynamic fields	Difficult	Easy with FormArray
Best for	Simple forms, quick prototypes	Complex forms, cross-field validation

TOPIC 4 OF 11 — PART 2

Angular Forms: Reactive Forms

Definition

A reactive form is an Angular form where the entire structure — every field, every validator, every rule — is built explicitly in the TypeScript class first, then connected to the HTML template. The template just binds to what already exists in the class. The form is a plain TypeScript object you can inspect, manipulate, and test without ever touching the DOM.

Diagram covered: reactive form architecture

TypeScript class (form built first): `FormBuilder.group({})` → `FormGroup` (the whole form) → `FormControl` × N → `Validators.required, .email` → `{ validators: passwordsMatch }`. HTML template (mirrors it): `[formGroup]="form"`, `formControlName="email"`, `(ngSubmit)="submit()"`, `*ngIf="email.hasError('email')"`.

The three building blocks

- **FormControl** — represents a single field. Holds value, validity state, and errors. One input = one FormControl.
- **FormGroup** — a collection of FormControls. Represents the whole form (or a sub-section).
- **FormArray** — a dynamic list of FormControls or FormGroups. Used for fields you add/remove at runtime.
- **FormBuilder** — a helper service that creates FormGroup/FormControl with less typing. `fb.group({...})` is shorthand for `new FormGroup({...})`.

Sub-topic 1: FormControl and FormGroup — your Login form

`login.component.ts (actual project code)`

```ts
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { finalize } from 'rxjs';
import { AuthService } from '../../../core/services/auth.service';

@Component({
  selector: 'app-login',
  standalone: false,
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent {
  form: FormGroup;
  loading  = false;
  error    = '';
  showPass = false;

  constructor(
    private fb: FormBuilder,
    private auth: AuthService,
    private router: Router
  ) {
    if (this.auth.isLoggedIn()) this.router.navigate(['/dashboard']);
    this.form = this.fb.group({
      email:    ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  get email()    { return this.form.get('email')!; }
  get password() { return this.form.get('password')!; }

  submit(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.loading = true;
    this.error   = '';

    this.auth.login(this.form.value).pipe(
      finalize(() => this.loading = false)
    ).subscribe({
      next: () => this.router.navigate(['/dashboard']),
      error: err => {
        this.error =
          err.error?.errors?.[0] ??
          err.error?.message     ??
          err.error?.error       ??
          'Invalid email or password. Please try again.';
      }
    });
  }

  togglePass(): void { this.showPass = !this.showPass; }
}
```

login.component.html

```
<form [formGroup]="form" (ngSubmit)="submit()" novalidate>
  <div class="form-group">
    <label for="email">Email Address</label>
    <input id="email" type="email"
      formControlName="email"
      [class.is-invalid]="email.invalid && email.touched" />
    <div class="field-error" *ngIf="email.touched && email.hasError('required')">
      Email is required.
    </div>
    <div class="field-error" *ngIf="email.touched && email.hasError('email')">
      Please enter a valid email address.
    </div>
  </div>

  <div class="form-group">
    <input id="password" [type]="showPass ? 'text' : 'password'"
      formControlName="password"
      [class.is-invalid]="password.invalid && password.touched" />
    <button type="button" (click)="togglePass()">{{ showPass ? 'Hide' : 'Show' }}</button>
    <div class="field-error" *ngIf="password.touched && password.hasError('minlength')">
      Password must be at least 6 characters.
    </div>
  </div>

  <button type="submit" [disabled]="loading">{{ loading ? 'Signing in…' : 'Sign In' }}</button>
</form>
```

Interactive covered: FormControl state at each stage

Stage 1 (form opened): value '', invalid, untouched, errors {required:true}, NO error shown. Stage 3 (tabs away empty): touched=true → error NOW shows. Stage 4 (types 'notanemail'): dirty=true, required gone, errors {email:true}, email error shows. Stage 5 (valid email): valid=true, errors null, no error. Stage 6 (submit empty): markAllAsTouched() forces touched=true on ALL controls → all errors appear.

Sub-topic 2: Built-in validators — your Register form

`register.component.ts (actual project code, constructor)`

```
this.form = this.fb.group(
  {
    name:     ['', [Validators.required, Validators.minLength(2), Validators.maxLength(200)]],
    email:    ['', [Validators.required, Validators.email, Validators.maxLength(256)]],
    dob:      ['', [Validators.required, this.dobValidator.bind(this)]],
    password: ['', [Validators.required, Validators.minLength(8), Validators.maxLength(128)]],
    confirm:  ['', Validators.required],
    phone:    ['', [Validators.maxLength(32), this.phoneValidator]],
  },
  { validators: this.passwordsMatch }   // cross-field validator on the whole group
);
```

Validator	Usage	Error key
Validators.required	Field must not be empty	required
Validators.email	Must be valid email format	email
Validators.minLength(n)	Min n characters	minlength
Validators.maxLength(n)	Max n characters	maxlength
Validators.min(n)	Numeric value ≥ n	min
Validators.max(n)	Numeric value ≤ n	max
Validators.pattern(regex)	Must match regex	pattern

Sub-topic 3: Custom validators — your dobValidator and phoneValidator

A custom validator is a function that receives an `AbstractControl` and returns either `null` (valid) or an error object (invalid).

`register.component.ts (actual project code)`

```
private dobValidator(control: AbstractControl): ValidationErrors | null {
  const value = control.value;
  if (!value) return null; // let required handle the empty case

  const dob   = new Date(value);
  const today = new Date();
  today.setHours(0, 0, 0, 0);

  if (dob >= today) return { dobFuture: true };

  const age18Cutoff = new Date(today);
  age18Cutoff.setFullYear(today.getFullYear() - 18);
  if (dob > age18Cutoff) return { dobMinAge: true };

  const age120Cutoff = new Date(today);
  age120Cutoff.setFullYear(today.getFullYear() - 120);
  if (dob < age120Cutoff) return { dobMaxAge: true };

  return null; // all checks passed
}

private phoneValidator(control: AbstractControl): ValidationErrors | null {
  const value = control.value?.trim();
  if (!value) return null; // optional field
  // ^[6-9] = starts with 6-9, \d{9} = 9 more digits, $ = end
  return /^[6-9]\d{9}$/.test(value) ? null : { phonePattern: true };
}
```

The template reads custom error keys exactly like built-in ones: `*ngIf="dob.touched && dob.hasError('dobFuture')"`, `dob.hasError('dobMinAge')`, `phone.hasError('phonePattern')`.

Sub-topic 4: Cross-field validator — passwordsMatch

A cross-field validator runs against the entire FormGroup instead of a single field. It's used when validation depends on two or more fields comparing against each other.

`register.component.ts (actual project code)`

```
private passwordsMatch(g: FormGroup) {
  const pw = g.get('password')?.value;
  const c  = g.get('confirm')?.value;
  // only error if BOTH have values and they don't match
  return pw && c && pw !== c ? { mismatch: true } : null;
}

get confirmError(): string {
  if (!this.confirm.touched) return '';
  if (this.confirm.hasError('required')) return 'Please confirm your password.';
  if (this.form.hasError('mismatch'))    return 'Passwords do not match.';
  return '';
}
```

The same pattern appears in `DashboardComponent` for the Change Password modal (`cpForm` with `currentPassword`, `newPassword`, `confirmPassword` and the same `passwordsMatch` validator).

Sub-topic 5: FormArray — dynamic fields (not in project, LifeTrack example)

`patient-symptoms.component.ts (hypothetical)`

```
this.form = this.fb.group({
  patientNotes: ['', Validators.required],
  symptoms: this.fb.array([
    this.fb.control('', Validators.required)  // first symptom field
  ])
});

get symptoms(): FormArray {
  return this.form.get('symptoms') as FormArray;
}
addSymptom(): void {
  this.symptoms.push(this.fb.control('', Validators.required));
}
removeSymptom(index: number): void {
  this.symptoms.removeAt(index);
}
```

```
<div formArrayName="symptoms">
  <div *ngFor="let symptom of symptoms.controls; let i = index">
    <input [formControlName]="i" placeholder="Enter symptom...">
    <button *ngIf="symptoms.length > 1" (click)="removeSymptom(i)">Remove</button>
  </div>
</div>
<button (click)="addSymptom()">+ Add Symptom</button>
```

Sub-topic 6: Setting and patching values programmatically

```
// setValue — sets ALL fields. Throws if you miss any.
this.form.setValue({ email: 'hari@lifetrack.com', password: '' });

// patchValue — sets only the fields you provide. Safe for partial updates.
this.form.patchValue({ email: 'hari@lifetrack.com' });

// reset — clears all fields and touched/dirty state (your CP modal does this)
this.cpForm.reset();

this.form.get('email')?.disable();   // disable programmatically
this.form.get('email')?.enable();    // re-enable
const emailValue = this.form.get('email')?.value;
```

Reactive forms interview reference

Question	Answer from your project
What is FormBuilder?	A service that creates FormGroup/FormControl with shorthand. fb.group() = new FormGroup()
setValue vs patchValue?	setValue requires ALL fields (throws if missing). patchValue is partial.
How handle cross-field validation?	Pass a validator function as 2nd arg to fb.group(): { validators: passwordsMatch }
What does markAllAsTouched() do?	Sets touched=true on every control, forcing all error messages to appear. Used in submit().
Custom vs built-in validator?	A function: takes AbstractControl, returns null (valid) or { errorKey: true } (invalid)
What is FormArray for?	Dynamic lists — push()/removeAt() add/remove fields at runtime

TOPIC 5 OF 11

Dependency Injection & Services

Definition

Dependency Injection (DI) is a design pattern where a class declares what it needs (its dependencies) and Angular's framework creates and supplies those objects automatically. You never write `new AuthService()` — you just ask for it in the constructor and Angular hands it to you. A **service** is a class decorated with `@Injectable` that holds logic meant to be shared across components.

Diagram covered: the DI injector concept

The Angular injector holds one `AuthService` instance. That same instance is handed to `LoginComponent`, `CtmDashboard`, and `AuthInterceptor` — all three share the exact same object in memory. When `currentUser$` emits a new value, every subscriber sees it instantly.

Sub-topic 1: Creating a service — @Injectable

```
@Injectable({ providedIn: 'root' })
export class AuthService { ... }

@Injectable({ providedIn: 'root' })
export class NavigationService { ... }

@Injectable({ providedIn: 'root' })
export class DashboardService { ... }
```

`providedIn: 'root'` means: "register this service in the root injector — make one instance available to the entire application." This is the correct default for almost every service.

Sub-topic 2: Your AuthService in depth (actual project code)

`core/services/auth.service.ts`

```ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable, tap } from 'rxjs';
import { Router } from '@angular/router';
import { environment } from '../../../environments/environment';
import { AuthResponse, LoginRequest, UserInfo } from '../models/auth.models';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly TOKEN_KEY = 'lt_token';
  private readonly USER_KEY  = 'lt_user';

  // Standard .NET role-claim URI emitted by ASP.NET Identity
  private readonly DOTNET_ROLE_CLAIM =
    'http://schemas.microsoft.com/ws/2008/06/identity/claims/role';

  // BehaviorSubject holds the CURRENT user and replays it to new subscribers
  private currentUserSubject = new BehaviorSubject<UserInfo | null>(this.getStoredUser());
  currentUser$ = this.currentUserSubject.asObservable();

  constructor(private http: HttpClient, private router: Router) {}

  login(req: LoginRequest): Observable<AuthResponse> {
    return this.http
      .post<AuthResponse>(`${environment.apiUrl}/auth/login`, req)
      .pipe(
        tap(res => {
          localStorage.setItem(this.TOKEN_KEY, res.token);
          localStorage.setItem(this.USER_KEY, JSON.stringify(res.user));
          this.currentUserSubject.next(res.user);  // broadcast to ALL subscribers
        })
      );
  }

  handleAuthResponse(res: AuthResponse): void {
    localStorage.setItem(this.TOKEN_KEY, res.token);
    localStorage.setItem(this.USER_KEY, JSON.stringify(res.user));
    this.currentUserSubject.next(res.user);
  }

  logout(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    localStorage.removeItem(this.USER_KEY);
    this.currentUserSubject.next(null);
    this.router.navigate(['/auth/login']);
  }

  getToken(): string | null {
    return localStorage.getItem(this.TOKEN_KEY);
  }

  isLoggedIn(): boolean {
    const token = this.getToken();
    if (!token) return false;
    const exp = this.getTokenExpiry();
    return exp ? exp > Date.now() : true;
  }

  get currentUser(): UserInfo | null {
    return this.currentUserSubject.value;   // synchronous read
  }

  // Reads from the JWT payload (tamper-resistant), NOT the cached user object
  private decodeToken(): Record<string, any> | null {
    const token = this.getToken();
    if (!token) return null;
    try {
```

```
      const parts = token.split('.');
      if (parts.length !== 3) return null;
      const payload = parts[1].replace(/-/g, '+').replace(/_/g, '/');
      const padded = payload + '='.repeat((4 - payload.length % 4) % 4);
      return JSON.parse(atob(padded));
    } catch { return null; }
  }

  getRoleFromToken(): string | null {
    const claims = this.decodeToken();
    if (!claims) return null;
    return claims['role'] ?? claims[this.DOTNET_ROLE_CLAIM] ?? null;
  }

  getTokenExpiry(): number | null {
    const claims = this.decodeToken();
    if (!claims?.['exp']) return null;
    return Number(claims['exp']) * 1000;   // JWT exp is seconds, JS uses ms
  }

  private getStoredUser(): UserInfo | null {
    try {
      const raw = localStorage.getItem(this.USER_KEY);
      return raw ? JSON.parse(raw) : null;
    } catch { return null; }
  }
}
```

Sub-topic 3: How services are injected — the constructor pattern

`ctm-dashboard.component.ts`

```
constructor(
  private auth:   AuthService,      // ← DI provides the root singleton
  private ds:     DashboardService, // ← DI provides the root singleton
  private router: Router,           // ← Angular's built-in Router
  private http:   HttpClient        // ← Angular's HTTP client
) {
  this.user = this.auth.currentUser;  // read current user synchronously
}
```

Sub-topic 4: DashboardService — a reusable HTTP service (actual project code)

`core/services/dashboard.service.ts`

```
const REQUEST_TIMEOUT_MS = 8000;

@Injectable({ providedIn: 'root' })
export class DashboardService {
  private readonly api = environment.apiUrl;
  constructor(private http: HttpClient) {}

  count(resource: string, params: Record<string, string> = {}): Observable<number> {
    const qs = new URLSearchParams({ page: '1', pageSize: '1', ...params });
    return this.http.get<PagedResult<unknown>>(`${this.api}/${resource}?${qs}`)
      .pipe(
        timeout(REQUEST_TIMEOUT_MS),       // throw if server >8s
        map(r => r?.totalCount ?? 0),     // extract just the count
        catchError(err => {                // any failure → 0
          console.error(`[Dashboard] count(${resource}) failed`, err);
          return of(0);
        })
      );
  }

  list<T>(resource: string, params: Record<string, string> = {}): Observable<T[]> {
    const qs = new URLSearchParams({ page: '1', pageSize: '6', ...params });
    return this.http.get<PagedResult<T>>(`${this.api}/${resource}?${qs}`)
      .pipe(
        timeout(REQUEST_TIMEOUT_MS),
        map(r => r?.items ?? []),
        catchError(err => {
          console.error(`[Dashboard] list(${resource}) failed`, err);
          return of([]);
        })
      );
  }
}
```

Sub-topic 5: NavigationService — stateful service (actual project code)

`core/services/navigation.service.ts`

```
@Injectable({ providedIn: 'root' })
export class NavigationService {
  private _previousUrl: string = '/dashboard';
  private _currentUrl:  string = '';

  constructor(private router: Router) {
    this.router.events
      .pipe(filter(e => e instanceof NavigationEnd))
      .subscribe((e: any) => {
        const next: string = (e as NavigationEnd).urlAfterRedirects;
        if (next !== this._currentUrl) {
          this._previousUrl = this._currentUrl || '/dashboard';
          this._currentUrl  = next;
        }
      });
  }

  get previousUrl(): string { return this._previousUrl; }

  back(fallback: string = '/dashboard'): void {
    const dest = this._previousUrl || fallback;
    this.router.navigate([dest]);
  }
}
```

Sub-topic 6: Hierarchical DI — three scopes

Scope	Declaration	Instances	LifeTrack example
Root	`@Injectable({ providedIn: 'root' })`	One for whole app	AuthService, DashboardService, NavigationService
Module	`providers: [Svc]` in @NgModule	One per module	Feature-specific caching (not used)
Component	`providers: [Svc]` in @Component	New per component	Multi-step wizard step state (not used)

Sub-topic 7: AuthInterceptor — a special injectable (actual project code)

`core/interceptors/auth.interceptor.ts`

```
@Injectable()   // No providedIn — manually registered in AppModule
export class AuthInterceptor implements HttpInterceptor {

  private static logoutInFlight = false;

  constructor(private auth: AuthService, private router: Router) {}

  intercept(req: HttpRequest<unknown>, next: HttpHandler): Observable<HttpEvent<unknown>> {
    const token = this.auth.getToken();
    const cloned = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;

    return next.handle(cloned).pipe(
      catchError((err: HttpErrorResponse) => {
        if (err.status === 401) {
          if (req.url.includes('/auth/login')) {
            return throwError(() => err);
          }
          if (!AuthInterceptor.logoutInFlight) {
            AuthInterceptor.logoutInFlight = true;
            this.auth.logout();
            queueMicrotask(() => { AuthInterceptor.logoutInFlight = false; });
          }
          return EMPTY;
        }
        return throwError(() => err);
      })
    );
  }
}
```

`app.module.ts`

```
@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,  // multi-token for interceptors
      useClass: AuthInterceptor,
      multi: true                  // REQUIRED — adds to array, doesn't replace
    }
  ]
})
export class AppModule {}
```

Sub-topic 8: Guards (actual project code)

`core/guards/auth.guard.ts`

```ts
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    this.router.navigate(['/auth/login']);
    return false;
  }
}
```

core/guards/role.guard.ts

```ts
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(route: ActivatedRouteSnapshot): boolean {
    if (!this.auth.isLoggedIn()) {
      this.router.navigate(['/auth/login']); return false;
    }
    const allowed: string[] = route.data?.['roles'] ?? [];
    if (allowed.length === 0) return true;
    const tokenRole = this.auth.getRoleFromToken();
    const userRole  = tokenRole ?? this.auth.currentUser?.role ?? '';
    if (allowed.includes(userRole)) return true;
    this.router.navigate(['/dashboard']);
    return false;
  }
}
```

DI INTERVIEW REFERENCE

providedIn:'root' = one singleton for whole app. **BehaviorSubject vs Subject** = BehaviorSubject replays last value to new subscribers. **Why read role from JWT** = localStorage is editable in DevTools; the JWT signature is server-verified. **Why static logoutInFlight flag** = forkJoin fires 6+ requests; if token expires all return 401; the flag ensures only the first calls logout().

TOPIC 6 OF 11

Angular Routing & Navigation

Definition

Angular's router maps URL paths to components. When the URL changes, the router destroys the current component and renders the matching one inside a `<router-outlet>` placeholder. No page reload happens. The browser history updates normally so the back button works.

Diagram covered: the LifeTrack route tree

`AppRoutingModule` (RouterModule.forRoot) branches to `/auth` (lazy, AuthModule, no guard → /auth/login, /auth/register) and `/dashboard` (lazy, DashboardModule, AuthGuard). Dashboard's shell DashboardComponent (topnav + router-outlet) branches to: '' → RoleHome (redirects by role), role dashboards (ctm/admin/patient…), shared feature pages (RoleGuard + data.roles), notifications. Feature routes like /adverse-events [CTM+INV+DM+RO], /documents [CTM+RO+DM], /audit-logs [Admin+RO].

Sub-topic 1: Configuring routes (actual project code)

app-routing.module.ts

```
const routes: Routes = [
  {
    path: 'auth',
    loadChildren: () =>
      import('./features/auth/auth.module').then(m => m.AuthModule)
  },
  {
    path: 'dashboard',
    loadChildren: () =>
      import('./features/dashboard/dashboard.module').then(m => m.DashboardModule),
    canActivate: [AuthGuard]   // runs BEFORE the lazy module loads
  },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: '**', redirectTo: 'dashboard' }   // wildcard — must be last
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],   // forRoot — only in AppRoutingModule
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Sub-topic 2: Nested routes — DashboardModule structure

dashboard.module.ts (route structure)

```
const routes: Routes = [
  {
    path: '',
    component: DashboardComponent,   // SHELL — topnav + <router-outlet>
    children: [
      { path: '', component: RoleHomeComponent, pathMatch: 'full' },
      { path: 'ctm',          component: CtmDashboardComponent },
      { path: 'admin',        component: AdminDashboardComponent },
      { path: 'investigator', component: InvestigatorDashboardComponent },
      { path: 'patient',      component: PatientDashboardComponent },
      { path: 'regulatory',   component: RegulatoryDashboardComponent },
      { path: 'data-manager', component: DataManagerDashboardComponent },
      {
        path: 'adverse-events',
        component: CtmAdverseEventsPageComponent,
        canActivate: [RoleGuard],
        data: {
          title: 'Adverse Events',
          roles: ['ClinicalTrialManager', 'Investigator', 'DataManager', 'RegulatoryOfficer']
        }
      },
      // ... more feature routes
    ]
  }
];

@NgModule({ imports: [RouterModule.forChild(routes)] })  // forChild in feature modules
export class DashboardModule {}
```

dashboard.component.html (the shell)

```
<div class="dashboard-wrapper">
  <header class="topnav">
    <span class="topnav-name">LifeTrack</span>
    <div class="topnav-right">
      <div class="user-chip">{{ user?.name }}</div>
      <button (click)="openCPModal()">Change Password</button>
      <button (click)="logout()">Sign Out</button>
    </div>
  </header>

  <router-outlet></router-outlet>   // child routes render HERE
</div>
```

Sub-topic 3: RoleHomeComponent — programmatic navigation (actual project code)

role-home.component.ts

```
const ROLE_ROUTES: Record<string, string> = {
  Admin:                '/dashboard/admin',
  ClinicalTrialManager: '/dashboard/ctm',
  Investigator:         '/dashboard/investigator',
  DataManager:          '/dashboard/data-manager',
  RegulatoryOfficer:    '/dashboard/regulatory',
  Patient:              '/dashboard/patient',
};

@Component({ selector: 'app-role-home', standalone: false, template: '' })
export class RoleHomeComponent implements OnInit {
  constructor(private auth: AuthService, private router: Router) {}
  ngOnInit() {
    const role   = this.auth.currentUser?.role ?? '';
    const target = ROLE_ROUTES[role];
    if (target) {
      this.router.navigate([target], { replaceUrl: true });
    }
  }
}
```

Programmatic navigation forms

```
this.router.navigate(['/dashboard/ctm']);                          // simple
this.router.navigate(['/dashboard/ctm'], { replaceUrl: true });      // replace history
this.router.navigate(['adverse-events'], { relativeTo: this.route }); // relative
this.router.navigate(['/dashboard/protocols'], { queryParams: { status: 'Active' } });
```

Sub-topic 4: Route parameters (LifeTrack example)

```
// Route: { path: 'protocols/:id', component: ProtocolDetailPageComponent }
ngOnInit(): void {
  const id = this.route.snapshot.paramMap.get('id');  // '42' read once
  this.loadProtocol(Number(id));

  // Or live updates when navigating 42 → 43 without recreation:
  this.route.paramMap.subscribe(params => {
    this.loadProtocol(Number(params.get('id')));
  });
}
```

Sub-topic 5: Lazy loading

Diagram covered: lazy loading sequence

Initial page load downloads main.js (AppModule + AppRoutingModule, Angular core, SharedModule, index.html) — NOT AuthModule or DashboardModule. First navigation to /dashboard: AuthGuard runs first (isLoggedIn?), then dashboard.module.js is downloaded, all dashboard components parsed, DashboardComponent renders. Next time: module already cached, navigation is instant.

```
{
  path: 'dashboard',
  loadChildren: () =>
    import('./features/dashboard/dashboard.module')  // separate bundle
      .then(m => m.DashboardModule),
  canActivate: [AuthGuard]
}
```

Sub-topic 6: routerLink — declarative navigation

```
<a routerLink="/dashboard/adverse-events">Adverse Events</a>
<a [routerLink]="['/dashboard/protocols', protocol.protocolID]">View Protocol</a>
<a routerLink="/dashboard/ctm" routerLinkActive="isActive">CTM Dashboard</a>
```

Your project uses button-click navigation: `manageAdverseEvents() { this.router.navigate(['/dashboard/adverse-events']); }`

Routing interview reference

Question	Answer
forRoot() vs forChild()?	forRoot creates the singleton Router — only in AppRoutingModule. forChild registers routes in feature modules.
What is router-outlet?	A placeholder where the matched child route's component renders.
What is lazy loading?	Splitting into separate JS bundles downloaded only when first visited. loadChildren triggers it.
pathMatch:'full'?	The entire URL must match, not just the start. Used on the '' redirect.
replaceUrl:true?	Replaces the current history entry. Used in RoleHomeComponent so Back skips the redirect.

TOPIC 7 OF 11

HTTP Client & APIs

Definition

Angular's `HttpClient` is the built-in service for making HTTP requests to backend APIs. Every request returns an `Observable` — you subscribe to get the response, and pipe it through RxJS operators to transform, catch errors, and combine multiple requests.

Setup — HttpClientModule in AppModule

`app.module.ts`

```
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';

@NgModule({
  imports: [ BrowserModule, HttpClientModule, AppRoutingModule ],
  providers: [
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
    { provide: DATE_PIPE_DEFAULT_OPTIONS, useValue: { timezone: '+0530' } }  // IST
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Diagram covered: the HTTP request pipeline

Component (`http.get(url)`) → AuthInterceptor (clones request, adds `Authorization: Bearer <jwt>`) → Ocelot gateway :5000 (routes to correct microservice) → Microservices (Auth :5001, User :5002, Protocol :5005, AE :5006, Visit :5007). Response flows back through the interceptor, where catchError handles 401 → logout().

Sub-topic 1: GET requests — your loadData() (actual project code)

`ctm-adverse-events-page.component.ts`

```
loadData(): void {
  this.loading = true;
  this.error   = '';
  const uid = this.authSvc.currentUser?.userID;

  forkJoin({
    protocols:     this.http.get<any>(`${environment.apiUrl}/protocols?pageSize=200`).pipe(catchError(() => of({
items: [] }))),
    patients:      this.http.get<any>(`${environment.apiUrl}/patients?pageSize=200`).pipe(catchError(() => of({
items: [] }))),
    events:        this.http.get<any>(`${environment.apiUrl}/adverse-events?pageSize=200`).pipe(catchError(() =>
of({ items: [] }))),
    siteProtocols: this.isInvestigator
      ? this.http.get<any>(`${environment.apiUrl}/site-protocols?
investigatorId=${uid}&pageSize=50`).pipe(catchError(() => of({ items: [] })))
      : of({ items: [] }),
    enrollments:   this.isInvestigator
      ? this.http.get<any>(`${environment.apiUrl}/enrollments?pageSize=200`).pipe(catchError(() => of({ items:
[] })))
      : of({ items: [] }),
  }).subscribe({
    next: ({ protocols, patients, events, siteProtocols, enrollments }) => {
      const allPatients  = patients.items  ?? [];
      const allProtocols = protocols.items ?? [];
      allPatients.forEach(p  => this.patientMap[p.patientID]   = p.name);
      allProtocols.forEach(p => this.protocolMap[p.protocolID] = p.title);
      this.adverseEvents = events.items ?? [];
      this.loading = false;
      this.applyFilter();
    },
    error: () => { this.error = 'Failed to load data.'; this.loading = false; }
  });
}
```

Sub-topic 2: POST — submitReport() (actual project code)

```
submitReport(): void {
  if (!this.validateReportForm()) return;
  this.reportSubmitting = true;
  const payload = {
    patientID:    +this.reportForm.patientID,   // + converts string → number
    protocolID:   +this.reportForm.protocolID,
    description:  this.reportForm.description.trim(),
    severity:     this.reportForm.severity,
    reportedDate: this.reportForm.reportedDate
  };
  this.http.post<{ eventID: number }>(`${environment.apiUrl}/adverse-events`, payload)
    .subscribe({
      next: (res) => {
        this.adverseEvents.push({ eventID: res.eventID, ...payload, status: 'Reported' } as AdverseEvent);
        this.applyFilter();
        this.reportSuccess = 'Adverse event reported successfully.';
        this.reportSubmitting = false;
      },
      error: () => { this.reportError = 'Failed to submit. Please try again.'; this.reportSubmitting = false; }
    });
}
```

Sub-topic 3: PUT — saveReviewStatus() (actual project code)

```
const payload = {
  patientID: this.reviewAE.patientID, protocolID: this.reviewAE.protocolID,
  description: this.reviewAE.description, severity: this.reviewAE.severity,
  status: this.reviewStatus, reportedDate: this.reviewAE.reportedDate
};
this.http.put<void>(`${environment.apiUrl}/adverse-events/${this.reviewAE.eventID}`, payload)
  .subscribe({
    next: () => {
      const original = this.adverseEvents.find(ae => ae.eventID === this.reviewAE!.eventID);
      if (original) Object.assign(original, payload);   // optimistic local update
      this.applyFilter();
      this.reviewSuccess = 'Status updated successfully.';
    },
    error: () => { this.reviewError = 'Failed to update status.'; }
  });
```

Sub-topic 4: DELETE — deleteNotification() (actual project code)

```
deleteNotification(n: any): void {
  this.http.delete(`${environment.apiUrl}/notifications/${n.notificationID}`)
    .pipe(takeUntil(this.destroy$))
    .subscribe({
      next: () => {
        this.notifications = this.notifications.filter((x: any) => x.notificationID !== n.notificationID);
      }
    });
}
```

Sub-topic 5: Observables vs Promises

Feature	Observable (HttpClient)	Promise (fetch)
Execution	Lazy — on subscribe	Eager — immediate
Cancellable	Yes (takeUntil)	No (needs AbortController)
Operators	map, catchError, forkJoin, etc.	.then() / .catch() only
Multiple emissions	WebSockets, streams	One value only

To convert: `const protocols = await firstValueFrom(this.http.get(...))`.

Sub-topic 6: RxJS operators in your project

Operator	What it does	In LifeTrack
map()	Transforms each value	DashboardService extracts totalCount/items
catchError()	Catches errors, returns fallback	Per-request of({items:[]}) in loadData forkJoin
takeUntil()	Auto-unsubscribe on notifier	takeUntil(destroy$) in CtmDashboard, NotificationsPage
tap()	Side effect, no value change	AuthService.login() stores token in localStorage
timeout()	Throws if no emit in N ms	DashboardService timeout(8000)
filter()	Passes only matching values	NavigationService filters NavigationEnd events
forkJoin()	Parallel, waits for all	All dashboard ngOnInit + AE loadData
finalize()	Runs on complete OR error	LoginComponent loading=false

Sub-topic 7: Error handling strategies

```
// Strategy 1: Silent fallback (DashboardService, loadData)
.pipe(catchError(() => of({ items: [] })))

// Strategy 2: User-visible error (submitReport, saveReviewStatus)
.subscribe({ error: () => { this.reportError = 'Failed to submit.'; } });

// Strategy 3: Global handler (AuthInterceptor)
catchError((err: HttpErrorResponse) => {
  if (err.status === 401) { this.auth.logout(); return EMPTY; }
  return throwError(() => err);
})
```

All HTTP calls in your project

Where	Method	URL
AuthService.login()	POST	/auth/login
RegisterComponent	POST	/auth/register/patient
DashboardComponent CP	POST	/auth/change-password
DashboardService.count()	GET	/{resource}?pageSize=1
DashboardService.list()	GET	/{resource}?pageSize=6
CtmAEPage.loadData()	GET ×5	protocols, patients, events, etc.
CtmAEPage.submitReport()	POST	/adverse-events
CtmAEPage.saveReviewStatus()	PUT	/adverse-events/:id
Notifications mark read	POST	/notifications/:id/read
Notifications delete	DELETE	/notifications/:id

DEEP DIVE (BETWEEN TOPICS 6 & 7)

Router Features: Guards, Events & Passing Data

The guard system — what it really is

A guard is a class that Angular calls before it commits to a navigation. Think of it as a checkpoint at a door — Angular asks "can you enter?", the guard says yes or no, and navigation either proceeds or gets cancelled/redirected.

Diagram covered: the four guard types on a navigation timeline

Step 1 — **CanDeactivate** (can we LEAVE this page?). Step 2 — **CanActivate** (can we ENTER the next page?). Step 3 — **Resolve** (pre-fetch data before render). Then the target page renders. Fail paths: CanDeactivate block → stay on current page; CanActivate block → redirect to login/dashboard. Purple (CanActivate) = guards in your project today; coral/teal (CanDeactivate/Resolve) = not yet but shown below.

Guard 1: CanActivate — your AuthGuard and RoleGuard

`CanActivate` runs before Angular loads a route component. It answers: "should the user be allowed to enter this page?" If it returns `false`, the component never renders.

core/guards/auth.guard.ts

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    this.router.navigate(['/auth/login']);
    return false;
  }
}
```

core/guards/role.guard.ts

```
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}
  canActivate(route: ActivatedRouteSnapshot): boolean {
    if (!this.auth.isLoggedIn()) {
      this.router.navigate(['/auth/login']); return false;
    }
    const allowed: string[] = route.data?.['roles'] ?? [];
    if (allowed.length === 0) return true;
    const tokenRole = this.auth.getRoleFromToken();
    const userRole  = tokenRole ?? this.auth.currentUser?.role ?? '';
    if (allowed.includes(userRole)) return true;
    this.router.navigate(['/dashboard']);
    return false;
  }
}
```

```
// Usage in routes
{
  path: 'adverse-events',
  component: CtmAdverseEventsPageComponent,
  canActivate: [RoleGuard],
  data: { title: 'Adverse Events', roles: ['ClinicalTrialManager', 'Investigator'] }
}
```

Guard 2: CanDeactivate — preventing accidental navigation away

Runs when the user tries to leave a page. Answers: "are you sure you want to leave?" Most common use: a form with unsaved changes. Not in your project yet — here's what it would look like for the AE Report modal.

core/guards/unsaved-changes.guard.ts

```
export interface CanComponentDeactivate {
  canDeactivate(): boolean | Observable<boolean>;
}

@Injectable({ providedIn: 'root' })
export class UnsavedChangesGuard implements CanDeactivate<CanComponentDeactivate> {
  canDeactivate(component: CanComponentDeactivate): boolean | Observable<boolean> {
    return component.canDeactivate();
  }
}
```

```
// In the component:
canDeactivate(): boolean {
  if (this.showReportModal && this.reportFormDirty) {
    return confirm('You have an unsaved adverse event report. Are you sure you want to leave?');
  }
  return true;
}

// On the route:
{ path: 'adverse-events', component: CtmAdverseEventsPageComponent,
  canActivate: [RoleGuard], canDeactivate: [UnsavedChangesGuard], data: {...} }
```

Guard 3: Resolve — pre-fetching data before the page renders

Runs after CanActivate passes but before the component loads. Fetches data and injects it into the route, so by the time `ngOnInit` runs, data is already available. No loading spinners.

core/resolvers/protocol.resolver.ts

```
@Injectable({ providedIn: 'root' })
export class ProtocolResolver implements Resolve<any> {
  constructor(private http: HttpClient) {}
  resolve(route: ActivatedRouteSnapshot): Observable<any> {
    const id = route.paramMap.get('id');
    return this.http.get(`${environment.apiUrl}/protocols/${id}`)
      .pipe(catchError(() => of(null)));
  }
}

// Route: resolve: { protocol: ProtocolResolver }
// Component: this.protocol = this.route.snapshot.data['protocol'];
```

	Resolve (pre-fetch)	ngOnInit fetch (your approach)
When data arrives	Before component renders	After component renders
Loading state needed?	No	Yes — loading=true
Best for	Detail pages with required data	List pages, dashboards

Router Events and Navigation Lifecycle

Interactive covered: router events lifecycle stepper

The full sequence Angular fires on every navigation: **NavigationStart** (navigation begins) → **RoutesRecognized** (route matched) → **GuardsCheckStart / GuardsCheckEnd** (guards running — your AuthGuard/RoleGuard fire here) → **ResolveStart / ResolveEnd** (resolvers running) → **NavigationEnd** (complete — your NavigationService subscribes here). Failure events: **NavigationCancel** (blocked by a guard) and **NavigationError** (resolver threw or lazy module failed to load).

navigation.service.ts (your code subscribing to NavigationEnd)

```
this.router.events
  .pipe(filter(e => e instanceof NavigationEnd))
  .subscribe((e: any) => {
    const next: string = (e as NavigationEnd).urlAfterRedirects;
    if (next !== this._currentUrl) {
      this._previousUrl = this._currentUrl || '/dashboard';
      this._currentUrl  = next;
    }
  });
// urlAfterRedirects is the FINAL URL after any redirects
// e.g. /dashboard → /dashboard/ctm (after RoleHome redirected)
```

Passing Data Between Routes — all four methods

Method	In project?	Persists refresh?	Use when
Route params (/protocols/:id)	Yes (pattern)	Yes	Data IS the page identity — an ID
Query params (?status=Active)	No	Yes	Optional filters, search, pagination
Static route data (data:{roles})	Yes	N/A	Fixed config — roles, title
Router state (navigation extras)	No	No	Temporary one-navigation data

1. Route parameters

```
this.router.navigate(['/dashboard/protocols', id]);   // → /dashboard/protocols/42
const id = this.route.snapshot.paramMap.get('id');    // read once
this.route.paramMap.subscribe(p => ...);              // live updates
```

2. Query parameters

```
this.router.navigate([], {
  relativeTo: this.route,
  queryParams: { severity: this.selectedSeverity || null, status: this.selectedStatus || null },
  queryParamsHandling: 'merge'
});
this.route.queryParamMap.subscribe(params => {
  this.selectedSeverity = params.get('severity') ?? '';
});
```

3. Static route data (your RoleGuard reads this)

```
{ path: 'adverse-events', component: ..., canActivate: [RoleGuard],
  data: { title: 'Adverse Events', roles: ['ClinicalTrialManager', 'Investigator'] } }

// In guard:  const allowed = route.data?.['roles'] ?? [];
// In component:  const title = this.route.snapshot.data['title'];
```

4. Router state (navigation extras)

```
this.router.navigate(['/dashboard'], { state: { successMessage: 'All notifications marked as read' } });
// Read in constructor of destination:
const nav = this.router.getCurrentNavigation();
const msg = nav?.extras?.state?.['successMessage'];
```

Router features interview reference

Question	Answer
CanActivate vs CanDeactivate?	CanActivate blocks entering a page. CanDeactivate blocks leaving one.
When use Resolve?	When a component always needs data to render. Pre-fetches before render, eliminating loading=true.
url vs urlAfterRedirects?	url is what was requested; urlAfterRedirects is the final URL after redirects (RoleHome /dashboard → /dashboard/ctm).
Multiple guards on one route?	Yes — canActivate: [AuthGuard, RoleGuard], run in order. Any false cancels navigation.
Pass success message page to page?	Router state: navigate(['/dashboard'], { state: { message } }), read via getCurrentNavigation().

DEEP DIVE (BETWEEN TOPICS 7 & 8)

AuthInterceptor — Detailed Workflow in LifeTrack

There are four completely distinct scenarios the interceptor handles. The interceptor is not a component, not a service you call — it is middleware that Angular inserts between your code and the network. Every single `http.get/post/put/delete` call passes through it automatically.

Diagram covered: interceptor position in the pipeline

Your code (`http.get(url)`) → outgoing → AuthInterceptor (1. add Bearer token, 2. catch 401 errors, 3. pass or swallow) → cloned + JWT added → Ocelot :5000 (routes to microservice) → Microservice :5001–:5008. Response travels back — the interceptor's catchError fires here on error.

The complete intercept() method — every line explained

`core/interceptors/auth.interceptor.ts (actual project code, fully annotated)`

```ts
@Injectable()
// No providedIn:'root' — manually registered in AppModule with HTTP_INTERCEPTORS
export class AuthInterceptor implements HttpInterceptor {

  // 'static' = belongs to the CLASS, shared across all instances.
  // forkJoin fires 7 calls simultaneously — if the token expires, all 7
  // return 401 at once. Without this flag, all 7 would call logout() in parallel,
  // creating a race on the router and localStorage.
  private static logoutInFlight = false;

  constructor(private auth: AuthService, private router: Router) {}

  intercept(
    req: HttpRequest<unknown>,
    next: HttpHandler
  ): Observable<HttpEvent<unknown>> {

    // Step 1: Read the JWT (null if not logged in)
    const token = this.auth.getToken();

    // Step 2: HttpRequest is IMMUTABLE. req.clone() creates a new request
    // with the Authorization header merged in. If no token, pass req unchanged.
    const cloned = token
      ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
      : req;

    // Step 3: Forward and pipe the response.
    return next.handle(cloned).pipe(
      catchError((err: HttpErrorResponse) => {

        if (err.status === 401) {

          // SPECIAL CASE: 401 on /auth/login = wrong password, NOT expiry.
          // Re-throw so LoginComponent shows "Invalid credentials".
          if (req.url.includes('/auth/login')) {
            return throwError(() => err);
          }

          // ALL OTHER 401s: token expired/revoked.
          // Static latch ensures only the FIRST of N parallel 401s acts.
          if (!AuthInterceptor.logoutInFlight) {
            AuthInterceptor.logoutInFlight = true;
            this.auth.logout();   // clears storage + navigates to /auth/login
            queueMicrotask(() => { AuthInterceptor.logoutInFlight = false; });
          }

          // EMPTY completes immediately without emitting. The component's
          // subscribe never fires — it's being destroyed by the navigation anyway.
          return EMPTY;
        }

        // 400, 403, 500, network — re-throw for the component to handle.
        return throwError(() => err);
      })
    );
  }
}
```

The four scenarios

Scenario 1 — Normal authenticated request (the 95% case)

1. CtmDashboard calls forkJoin → `this.ds.count('protocols')` → `http.get`
2. `intercept()` fires automatically before any network activity
3. `auth.getToken()` returns the JWT

4. `req.clone()` adds `Authorization: Bearer eyJ...`
5. `next.handle(cloned)` sends GET with the header
6. Ocelot validates JWT, routes to Protocol service, returns 200 OK
7. catchError does NOT fire — 200 passes straight through
8. forkJoin collects all results → `this.protocols = 12` → `loading = false`

Scenario 2 — Public route (login/register), no token

1. LoginComponent calls `auth.login()` → `http.post('/auth/login', {...})`
2. `intercept()` fires — same code path
3. `auth.getToken()` returns `null` (not logged in yet)
4. Ternary takes else branch — `cloned = req` (unmodified, NO Authorization header)
5. Backend processes login, returns 200 + JWT
6. `auth.login()`'s `tap()` stores token + user
7. LoginComponent navigates to /dashboard

Scenario 3 — Wrong password: 401 on /auth/login

1. Backend returns 401 Unauthorized (wrong password)
2. catchError fires, `err.status === 401`
3. `req.url.includes('/auth/login')` → true → special case detected
4. `return throwError(() => err)` — re-throws to the caller
5. LoginComponent error handler: `this.error = err.error?.message ?? 'Invalid email or password.'`
6. User sees the error, stays on login page — NO logout, NO redirect

Scenario 4 — Session expired: 401 on dashboard (the race condition)

1. CtmDashboard ngOnInit fires forkJoin with 7 parallel requests
2. All 7 pass through intercept(), each cloned with the (now expired) JWT
3. All 7 return 401 simultaneously — all 7 catchError handlers fire within ms
4. Request #1 reaches the 401 branch first → `!logoutInFlight` is true
5. Sets `logoutInFlight = true` → calls `auth.logout()` ONCE (clears storage, navigates)
6. `queueMicrotask` schedules the flag reset after sync code finishes
7. Requests #2–7 reach the 401 branch → `logoutInFlight` is now true → skip the if block
8. All 7 return EMPTY — no errors, no double-navigation. User sees login page once.

> **Diagram covered: race condition — without vs with the static flag**
>
> **Without the flag (the bug):** 7 requests all return 401 → 7 × auth.logout() + 7 × router.navigate('/auth/login') → race condition crash. **With the flag (the fix):** req #1 401 → logout() once; reqs #2–7 → return EMPTY. 1 logout, 1 navigation, no race. User sees /auth/login once.

Why req.clone() is necessary — immutability

```
// WRONG — throws a runtime error
req.headers.set('Authorization', `Bearer ${token}`);

// WRONG — silently does nothing (append returns a new object, doesn't mutate)
req.headers.append('Authorization', `Bearer ${token}`);

// CORRECT — clone() creates a new HttpRequest with changes merged in
const cloned = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
```

Angular makes requests immutable because the same `req` object can be read by multiple interceptors in the chain. Immutability guarantees each interceptor gets a clean copy.

EMPTY vs throwError — why we use both

```
// EMPTY — completes without emitting. subscriber's next()/error() NEVER fire.
// Used for 401 non-login errors — the component is being destroyed anyway.
return EMPTY;

// throwError — re-throws to the subscriber. error() handler fires.
// Used for 401 on login, and all 400/403/500 errors.
return throwError(() => err);
```

How the interceptor is registered — multi:true

```
providers: [
  {
    provide: HTTP_INTERCEPTORS,   // multi-provider token = an array slot
    useClass: AuthInterceptor,
    multi: true                   // ADD to the array, don't replace it
  }
]
// Without multi:true, your interceptor REPLACES the entire array,
// losing any other interceptors (e.g. a logging interceptor).
```

queueMicrotask vs setTimeout

```
// queueMicrotask (your code) — runs immediately after current sync code,
// before the next macrotask. All 7 catchError handlers run synchronously,
// so by the time the microtask fires, all 7 have checked the flag. No race window.
queueMicrotask(() => { AuthInterceptor.logoutInFlight = false; });
```

Summary — the four decision paths

```
Every http.get/post/put/delete
        ↓
intercept() fires automatically
        ↓
getToken() ── null ──→ send req unchanged (login/register)
     ↓
    JWT ──→ req.clone({ Authorization: Bearer JWT })
                ↓
         next.handle(cloned) sends request
                ↓
         Response comes back
                ↓
       status 200-299 ──→ pass through → component receives data
                ↓
       status 401 ──→ url is /auth/login? ── YES → throwError → LoginComponent shows message
                                          ── NO  → logoutInFlight? ── YES → return EMPTY (skip)
                                                                     ── NO  → set flag → logout() → EMPTY
                ↓
       status 400/403/500 ──→ throwError → component error handler fires
```

This is why no component in your project ever needs to check for an expired session or handle 401 errors — the interceptor handles it all globally, invisibly, for every request simultaneously.

TOPIC 8 OF 11

State Management in Angular

Definition

State management is how an application decides where to store data, who can read it, who can change it, and how changes propagate to every part of the UI that cares. Without a strategy, every component independently fetches the same data, components go out of sync, and debugging becomes nearly impossible.

Diagram covered: three layers of state in LifeTrack

Server state (outermost) — lives in SQL Server + microservices; adverse events, protocols, patients, enrollments — fetched via HTTP on demand. **Shared service state** — lives in singleton services; currentUser, previousUrl, notifications — shared across ALL components instantly (AuthService currentUser$, NavigationService previousUrl, DashboardService count/list). **Local component state** — lives inside each component; loading, error, searchTerm, showModal, filteredEvents — dies when the component is destroyed.

Sub-topic 1: Local component state

Plain TypeScript class properties declared directly in a component. They exist only while that component is alive and are invisible to every other component.

`ctm-adverse-events-page.component.ts (all local state)`

```
export class CtmAdverseEventsPageComponent implements OnInit {
  // Data state — loaded from API, stays local
  adverseEvents: AdverseEvent[] = [];
  filteredEvents: AdverseEvent[] = [];   // derived from adverseEvents
  protocolMap: { [id: number]: string } = {};
  patientMap:  { [id: number]: string } = {};

  // Loading/error state
  loading = false;
  error   = '';
  success = '';

  // Filter state — drives the search/filter bar
  selectedStatus   = '';
  selectedSeverity = '';
  searchTerm       = '';

  // Modal state
  showReviewModal  = false;
  showReportModal  = false;
  reviewAE: AdverseEvent | null = null;
  reviewStatus     = '';

  // Role flags — derived once in ngOnInit
  isInvestigator      = false;
  isRegulatoryOfficer = false;
  isDataManager       = false;
}
```

`filteredEvents` is derived entirely from `adverseEvents` plus the filter values. `applyFilter()` rebuilds a local array on every filter change — no service, no global store.

Sub-topic 2: Shared state via Services — BehaviorSubject

The pattern: a service holds the state in a `BehaviorSubject`, exposes it as a public Observable, and components subscribe to get live updates.

`core/services/auth.service.ts`

```
// BehaviorSubject = a box that holds a value AND has a loudspeaker
private currentUserSubject = new BehaviorSubject<UserInfo | null>(
  this.getStoredUser()   // initialise from localStorage on startup
);

// .asObservable() removes .next() — only AuthService can push values
currentUser$ = this.currentUserSubject.asObservable();

// Synchronous getter for when you need the value RIGHT NOW
get currentUser(): UserInfo | null {
  return this.currentUserSubject.value;
}
```

```
// login() pushes new state — broadcasts to ALL subscribers at once
this.currentUserSubject.next(res.user);
// logout() resets to null — same broadcast mechanism
this.currentUserSubject.next(null);
```

Three consumption patterns

```
// Pattern A: synchronous read in constructor (DashboardComponent, CtmDashboard)
this.user = this.auth.currentUser;

// Pattern B: read once in ngOnInit (CtmAdverseEventsPage)
const user = this.authSvc.currentUser;
this.isInvestigator = user?.role === 'Investigator';

// Pattern C: subscribe to reactive stream (for live updates)
this.auth.currentUser$.pipe(takeUntil(this.destroy$))
  .subscribe(user => { this.user = user; });
```

Diagram covered: Subject vs BehaviorSubject

Subject (no memory): login() emits the user, but CtmDashboard subscribes AFTER the emit → receives NOTHING, user is null.
BehaviorSubject (replays current value): login() emits, CtmDashboard subscribes AFTER → immediately receives the stored user value.
BehaviorSubject remembers its last emitted value and gives it to any new subscriber.

Sub-topic 3: NavigationService — simple mutable state

```
@Injectable({ providedIn: 'root' })
export class NavigationService {
  private _previousUrl = '/dashboard';   // plain mutable state
  private _currentUrl  = '';
  // updates itself by listening to router events; no Observable needed
  // since no component reactively subscribes — they call nav.back() once
}
```

Sub-topic 4: Unread notification count — derived state (current limitation)

```
// CURRENT: each component fetches independently — can get out of sync
// CtmDashboardComponent AND NotificationsPageComponent both GET /notifications/my

// IMPROVED: a shared NotificationStateService with BehaviorSubject
@Injectable({ providedIn: 'root' })
export class NotificationStateService {
  private notifications$ = new BehaviorSubject<Notification[]>([]);
  readonly all$         = this.notifications$.asObservable();
  readonly unreadCount$ = this.notifications$.pipe(
    map(ns => ns.filter(n => n.status === 'Unread').length)
  );
  markRead(id: number): void {
    const updated = this.notifications$.value.map(n =>
      n.notificationID === id ? { ...n, status: 'Read' } : n);
    this.notifications$.next(updated);   // broadcasts to ALL subscribers
  }
}
```

Sub-topic 5: NgRx — advanced state management (not in project)

NgRx is Angular's Redux-style state management. One single Store holds all state. You dispatch Actions, Reducers process them to produce a new state, Selectors extract pieces.

```
// Conceptual NgRx in LifeTrack
interface AeState { events: AdverseEvent[]; loading: boolean; error: string | null; }

const loadAEs        = createAction('[AE] Load');
const loadAEsSuccess = createAction('[AE] Load Success', props<{ events: AdverseEvent[] }>());
const updateAeStatus = createAction('[AE] Update Status', props<{ id: number; status: string }>());

const aeReducer = createReducer(initialState,
  on(loadAEs,        state => ({ ...state, loading: true })),
  on(loadAEsSuccess, (state, { events }) => ({ ...state, events, loading: false })),
  on(updateAeStatus, (state, { id, status }) => ({
    ...state,
    events: state.events.map(ae => ae.eventID === id ? { ...ae, status } : ae)
  }))
);

// Component: this.store.dispatch(loadAEs()); events$ = this.store.select(selectFiltered);
```

Tradeoff: NgRx is much more code but gives time-travel debugging and strict unidirectional flow. LifeTrack doesn't need it — `BehaviorSubject` is sufficient.

When to use which approach

Scenario	Approach	LifeTrack example
Data only one component needs	Local component state	filteredEvents, showReviewModal
Data that survives navigation	localStorage	JWT token, escalated AE IDs
Data needed by 2+ components	Shared service + BehaviorSubject	currentUser$ in AuthService
Derived read-only values	Service getter / map() pipe	isLoggedIn(), unreadCount$
Large app, many teams	NgRx Store	Not in LifeTrack yet

State management interview reference

Question	Answer from your project
What is a BehaviorSubject?	A Subject that remembers its last value and replays it to new subscribers. AuthService uses it so components created after login still get the user.
Why use .asObservable()?	To hide .next() from consumers. Only AuthService should push values to currentUser$.
currentUser getter vs currentUser$?	currentUser is synchronous (value now, for constructors). currentUser$ is an Observable (react to future changes).
Why no NgRx in LifeTrack?	State is simple and role-based with minimal cross-module sharing. BehaviorSubject handles the only shared state. NgRx would add boilerplate for little benefit.
Danger of local state?	It's destroyed with the component. Navigating away and back loses filteredEvents — must re-fetch. Fine for lists, bad for forms mid-completion.

TOPIC 9 OF 11

Reactive Programming with RxJS

Definition

RxJS (Reactive Extensions for JavaScript) is a library for composing asynchronous and event-based programs using **Observables** — streams of values that arrive over time. Instead of writing imperative "do this, then that" code, you describe what should happen to data as it flows through a pipeline of operators.

Diagram covered: Observable stream mental model

Picture a conveyor belt. `http.get()` Observable emits AE[] values over time. Each operator is a worker along the belt: `map(r => r.items)` (PagedResult → AE[]), `timeout(8000)` (throws if >8s), `catchError` (errors → of([])). The subscriber at the end (`.subscribe()` → `this.events = data`) does something with the finished product. The belt doesn't move until someone subscribes.

Sub-topic 1: Observable and Observer

```
this.http.get<any>(`${environment.apiUrl}/adverse-events?pageSize=200`)
  .subscribe({
    next: (res) => { this.adverseEvents = res.items ?? []; this.loading = false; },
    error: (err) => { this.error = 'Failed to load adverse events.'; this.loading = false; },
    complete: () => { console.log('Stream completed'); }
  });
```

The Observer's three callbacks: `next` (each emitted value), `error` (stream terminates), `complete` (finishes normally). For HTTP, next fires once then complete.

Sub-topic 2: Subject and BehaviorSubject

```
// 1. Subject<void> — the destroy$ pattern
private destroy$ = new Subject<void>();
// used as a stop signal for takeUntil; ngOnDestroy: destroy$.next(); destroy$.complete();

// 2. BehaviorSubject<UserInfo|null> — the auth state
private currentUserSubject = new BehaviorSubject<UserInfo | null>(this.getStoredUser());
this.currentUserSubject.next(res.user);     // push new value
const u = this.currentUserSubject.value;     // read synchronously
```

Sub-topic 3: Creation operators

```
// of() — emits the value(s) immediately and completes (your catchError fallbacks)
catchError(() => of({ items: [] }))
catchError(err => { console.error(...); return of(0); })

// EMPTY — completes immediately with NO value (AuthInterceptor swallows 401)
return EMPTY;

// from() — converts Promise/array/iterable into an Observable
from([1, 2, 3]).subscribe(n => console.log(n));

// interval() — emits incrementing numbers at a time interval
interval(30000).pipe(takeUntil(this.destroy$)).subscribe(() => this.loadData());
```

Sub-topic 4: Combination operators — forkJoin

```
forkJoin({
  // All requests fire SIMULTANEOUSLY at T=0ms
  protocols: this.http.get<any>(`${api}/protocols`).pipe(catchError(() => of({ items: [] }))),
  patients:  this.http.get<any>(`${api}/patients`).pipe(catchError(() => of({ items: [] }))),
  events:    this.http.get<any>(`${api}/adverse-events`).pipe(catchError(() => of({ items: [] }))),
})
// emits once when the SLOWEST request finishes
.subscribe({ next: ({ protocols, patients, events }) => { ... } });
```

Other combination operators: `combineLatest` (emits when ANY source emits, with latest from each), `merge` (interleaved), `switchMap` (cancels previous inner Observable when new outer arrives — essential for search-as-you-type).

Sub-topic 5: Pipeable operators — full project inventory

Operator	In project?	What it does / LifeTrack usage
tap()	Yes	Side effect, no value change. AuthService.login() stores token in localStorage.
map()	Yes	Transforms each value. DashboardService.count() pulls totalCount from PagedResult.
catchError()	Yes	Catches errors, returns fallback. Per-request of({items:[]}) in loadData forkJoin.
takeUntil()	Yes	Auto-unsubscribe on notifier. takeUntil(destroy$) in CtmDashboard, NotificationsPage.
timeout()	Yes	Throws if no emit in N ms. DashboardService timeout(8000).
filter()	Yes	Passes only matching values. NavigationService filters NavigationEnd events.
finalize()	Yes	Runs on complete OR error. LoginComponent loading=false.
debounceTime()	No	Emits after N ms of silence. For search-as-you-type (not yet used).
distinctUntilChanged()	No	Suppresses consecutive duplicates. Pairs with debounceTime.
switchMap()	No	Cancels previous inner Observable when new outer arrives. Server-side search.

tap() — actual project code

```
login(req: LoginRequest): Observable<AuthResponse> {
  return this.http.post<AuthResponse>(`${api}/auth/login`, req)
    .pipe(
      tap(res => {
        localStorage.setItem('lt_token', res.token);
        localStorage.setItem('lt_user', JSON.stringify(res.user));
        this.currentUserSubject.next(res.user);
        // subscriber still receives the full AuthResponse unchanged
      })
    );
}
```

takeUntil(destroy$) — the complete pattern

```
private destroy$ = new Subject<void>();
ngOnInit() {
  this.http.get<any>(`${api}/notifications/my`)
    .pipe(catchError(() => of({ items: [] })), takeUntil(this.destroy$))
    .subscribe(res => this.notifications = res.items ?? []);
}
ngOnDestroy(): void {
  this.destroy$.next();      // fires → ALL takeUntil subscriptions complete
  this.destroy$.complete();  // closes the Subject
}
```

Sub-topic 6: The four reactive patterns in LifeTrack

Pattern 1 — Safe parallel loading: `forkJoin` + per-request `catchError`. Five HTTP calls fire simultaneously, each protected so one failure doesn't abort all.

Pattern 2 — Memory-safety: `takeUntil(destroy$)` + `ngOnDestroy`. One `destroy$.next()` kills all subscriptions.

Pattern 3 — State broadcasting: `BehaviorSubject` in AuthService. One service pushes state, many consumers subscribe.

Pattern 4 — Router event filtering: NavigationService filters `NavigationEnd` from the mixed-type `router.events` stream.

Diagram covered: RxJS usage map in LifeTrack

auth.service.ts → BehaviorSubject, tap(), Observable. dashboard.service.ts → map(), catchError(), timeout(), of(). ctm-dashboard.ts → forkJoin, takeUntil(), Subject, catchError(). ctm-ae-page.ts → forkJoin, catchError(), of(). navigation.service.ts → filter(), Router$. login.component.ts → finalize().

RxJS interview reference

Question	Answer
What is an Observable?	A lazy stream of values over time. http.get() doesn't fire until .subscribe().
Subject vs BehaviorSubject?	Subject has no memory; late subscribers miss past emissions. BehaviorSubject replays the last value.
Why takeUntil over manual unsubscribe?	One destroy$.next() unsubscribes ALL at once, regardless of count.
What is forkJoin?	Runs Observables in parallel, waits for ALL to complete, emits one combined result.
What does of() do?	Creates an Observable that immediately emits the value(s). Used as catchError fallbacks.
What is switchMap for?	Cancels the previous inner Observable when a new outer arrives. Essential for search-as-you-type.
Forget takeUntil?	Subscription outlives the component — 5 navigations = 5 active subscriptions updating a destroyed component. Memory leak.

SECTION 11

Interview Questions & Answers (Q1–Q55)

Each answer is grounded in the LifeTrack project where the concept exists, otherwise a general example is given. Tags: `in project` *= drawn from actual LifeTrack code;* `general` *= general Angular concept.*

Batch 1: Q1–Q25

Q1 What is a Routing Guard in Angular? in project

A routing guard is a class Angular calls before activating/deactivating a route. It returns true to allow or false/redirect to block. Your project has two: `AuthGuard` (checks `auth.isLoggedIn()`, redirects to /auth/login, applied to the whole /dashboard route) and `RoleGuard` (reads `route.data.roles`, compares against `auth.getRoleFromToken()`; a CTM accessing /dashboard/admin/users is redirected to /dashboard).

Q2 What is a module, and what does it contain? in project

A container grouping related components, services, directives, pipes. Declared with `@NgModule`. Your project: `AppModule` (root, bootstraps AppComponent, registers AuthInterceptor), `AuthModule` (Login + Register, imports ReactiveFormsModule), `DashboardModule` (all dashboard components, lazy loaded). The four key arrays: **declarations** (components/directives/pipes owned), **imports** (other modules whose exports you need), **providers** (services scoped to this module), **exports** (declarations other modules can use).

Q3 What are pipes? Give an example. in project

A pipe transforms a value in a template without mutating the original. Syntax: `{{ value | pipeName:argument }}`. Your project: `{{ ae.reportedDate | date:'mediumDate' }}` → "May 14, 2026"; `{{ n.createdDate | date:'MMM d, y · h:mm a' }}` → "May 14, 2026 · 8:30 AM"; `{{ r.enrollmentRate | number:'1.1-1' }}` → "87.3".

Q4 What is a component? Why would you use it? in project

A component controls a patch of screen (a view) — a TypeScript class + HTML template + optional CSS. Used to break a large UI into small, reusable pieces. Your project: `DashboardComponent` (shell with topnav + router-outlet), `CtmDashboardComponent` (KPI cards), `CtmAdverseEventsPageComponent` (AE table, modals, filters).

Q5 Difference between an Angular component and a module? general

A component is a single UI building block (one view). A module is an organiser grouping components, directives, pipes, services. Analogy: modules are folders, components are files. In your project: `DashboardModule` contains all dashboard components plus routes — the module organises, each component is a UI piece.

Q6 What is a service, and when will you use it? in project

A class with `@Injectable` holding business logic, HTTP, or shared state. Use it when logic is reused across components or state must persist across navigation. Your project: `AuthService` (login/logout, JWT, current user via BehaviorSubject), `DashboardService` (reusable count()/list() with timeout+catchError), `NavigationService` (tracks previousUrl).

Q7 Equivalent of ngShow and ngHide in Angular? in project

AngularJS ng-show/ng-hide → Angular uses `*ngIf` (removes from DOM entirely) or `[hidden]` (display:none, keeps in DOM). Your project uses *ngIf everywhere: `<button *ngIf="isInvestigator">`, `<div *ngIf="loading">`, `<div *ngIf="showReviewModal">`. Prefer *ngIf for performance.

Q8 How can I select an element in a component template? general

Use `@ViewChild` with a template reference variable (#). Available in `ngAfterViewInit()`. Example: `@ViewChild('kpiChart') chartCanvas!: ElementRef;` then `this.chartCanvas.nativeElement.getContext('2d')`. Your CtmReportsPage uses `[attr.stroke]` / `[style]` bindings on SVG instead — Angular's declarative approach.

Q9 Difference between @Component and @Directive? general

`@Component` has a template (visible UI). `@Directive` has NO template — it modifies an existing element's behaviour/appearance, applied as an attribute. Rule: if it needs HTML, use @Component; if it just changes an existing element, use @Directive.

Q10 How would you protect a component being activated through the router? in project

Add `canActivate: [GuardClass]` to the route. Your project: two-layer protection. Layer 1 — AuthGuard on the parent /dashboard route (must be logged in). Layer 2 — RoleGuard on individual pages with `data: { roles: [...] }` (must have the right role).

Q11 How would you run unit tests? general

`ng test` starts Karma + Jasmine, runs all .spec.ts files in Chrome. `ng test --no-watch --code-coverage` for a single run with coverage. Every generated component has a .spec.ts file.

Q12 Difference between structural and attribute directives? in project

Structural (prefixed *) add/remove DOM elements: `*ngIf`, `*ngFor`, `*ngSwitch`. **Attribute** change appearance/behaviour without adding/removing: `[ngClass]`, `[ngStyle]`, `[(ngModel)]`. Your project: `*ngFor="let ae of filteredEvents"` (structural), `[ngClass]="severityClass(ae.severity)"` (attribute).

Q13 Purpose of the base href tag? general

`<base href="/">` in index.html tells the browser the base URL for relative URLs. The router uses it to construct full URLs. For a subdirectory deploy, set `<base href="/myapp/">` or pass `--base-href` to ng build.

Q14 What is an observer? in project

The object passed to `.subscribe()`. Three callbacks: `next(value)`, `error(err)`, `complete()`. Your submitReport(): `.subscribe({ next: (res) => {...}, error: () => {...} })`. complete not used — HTTP completes automatically after next().

Q15 What is an observable? in project

A lazy stream of values over time. Nothing happens until you subscribe. Lazy, cancellable, composable via .pipe(). Your AuthService.login() returns an Observable — nothing fires until LoginComponent calls .subscribe().

Q16 What are observables? (expanded) in project

The core data type of RxJS — streams of async events. Three types in your project: **Cold** (http.get() — each subscriber triggers a new request), **BehaviorSubject** (currentUser$ — shared, replays last value), **Router events** (shared stream NavigationService subscribes to).

Q17 What is a bootstrapping module? in project

The root module Angular loads first, passed to `platformBrowserDynamic().bootstrapModule()` in main.ts. Your project: `AppModule` bootstraps AppComponent which renders `<app-root>`.

Q18 What is interpolation? in project

The `{{ }}` syntax embedding a TS expression into HTML as text. One-way (class → template). Your project: `{{ ae.description }}`, `{{ i + 1 }}`, `{{ reportSubmitting ? 'Submitting...' : 'Submit AE' }}`, `{{ patientMap[ae.patientID] || 'Patient #' + ae.patientID }}`.

Q19 Difference between Promise and Observable? in project

Promise: eager, not cancellable, one value, .then() only. Observable: lazy (fires on subscribe), cancellable (unsubscribe), zero-to-many values, rich operators. In LifeTrack, all HTTP returns Observables; `takeUntil(destroy$)` cancels in-flight requests; `forkJoin` fires 7 in parallel — impossible with Promises.

Q20 Difference between constructor and ngOnInit? in project

constructor — JS instantiation; use ONLY for DI; no inputs set, no template. **ngOnInit** — after @Input() set; right place for HTTP, subscriptions, setup. Your pattern: constructor injects AuthService/DashboardService/Router/HttpClient and reads `this.auth.currentUser` synchronously; ngOnInit fires the forkJoin.

Q21 Why use ngOnInit if we have a constructor? general

1. Inputs aren't set in the constructor — ngOnInit runs after. 2. Separation of concerns — constructor for DI, ngOnInit for init logic. 3. Testability — you can construct and inspect before calling ngOnInit().

Q22 How to bundle an Angular app for production? general

`ng build --configuration production`: AOT compilation, tree shaking, minification, optional source maps. Output to /dist/. For LifeTrack, copy /dist/ to IIS/Azure Static Apps — all API calls go to the Ocelot gateway.

Q23 Difference between declarations, providers and imports in NgModule? in project

declarations — components/directives/pipes that BELONG to this module. **imports** — other modules whose exports you USE (e.g. ReactiveFormsModule for [formGroup]). **providers** — services for this module's injector scope. Your AuthModule: declarations [LoginComponent, RegisterComponent], imports [CommonModule, ReactiveFormsModule, RouterModule.forChild].

Q24 What is Reactive Programming and how to use it with Angular? in project

A paradigm modelling data as streams over time, composing transformations rather than imperative code. RxJS provides the tools. Your project: `forkJoin({...}).pipe(takeUntil(destroy$)).subscribe(...)` declares what happens when data arrives; `currentUserSubject.next(user)` broadcasts state changes reactively to all subscribers.

Q25 Why would you use a spy in a test? general

A Jasmine spy wraps a function and records all calls. Use it to: isolate the component from real services (replace AuthService.login() with a spy returning a fake Observable), assert behaviour (verify router.navigate() was called), control return values. Example: `jasmine.createSpyObj('AuthService', ['login']); authSpy.login.and.returnValue(of({...}));`

Batch 2: Q26–Q55

Q26 What is TestBed? general

Angular's primary testing utility — creates a miniature Angular module in memory to test components/services with real DI and change detection, without a browser. `TestBed.configureTestingModule({...}); const fixture = TestBed.createComponent(...); fixture.detectChanges();`

Q27 What is Protractor? general

Angular's old E2E framework on Selenium WebDriver. Deprecated in Angular 12 and sunsetted. Modern replacements: Cypress or Playwright. A Cypress test for LifeTrack would open a browser, navigate to /auth/login, fill the form, submit, assert the CTM dashboard renders.

Q28 Point of renderer.invokeElementMethod()? general

An old Renderer (v1) method for calling native DOM methods safely across platforms; removed in Renderer2. Modern Angular uses Renderer2: `this.renderer.addClass(this.el.nativeElement, 'highlight-critical')`. Reason to use Renderer2 over direct nativeElement: works in SSR and web workers.

Q29 How to control element size on window resize? general

Use `@HostListener('window:resize', ['$event'])` to update component properties that drive template bindings. Your KPI gauges use percentage-based `[style.width.%]` + SVG viewBox with width="100%" — which avoids resize listeners since SVG scales automatically.

Q30 What is Redux and how does it relate to an Angular app? in project

A state pattern where all state lives in one central immutable store, changed only via dispatched actions processed by pure reducers. NgRx is Redux for Angular. LifeTrack uses service-based state (not NgRx) — AuthService's BehaviorSubject is a mini-store for the current user, sufficient for its scale.

Q31 When would you use eager module loading? in project

When a module is needed on the first page load — included in the initial bundle. Your project eager-loads AppModule (root), HttpClientModule, BrowserModule. Lazy-loads AuthModule and DashboardModule (behind navigation/login). Rule: needed on first load → eager; behind navigation → lazy.

Q32 Core dependencies of Angular? general

`@angular/core`, `@angular/common`, `@angular/forms`, `@angular/router`, `@angular/platform-browser`, `@angular/common/http`, `rxjs`, `zone.js`, `typescript`. Your package.json includes all plus `@angular/animations`.

Q33 Why does Incremental DOM have a low memory footprint? general

Angular Ivy's Incremental DOM does NOT create a virtual DOM tree in memory — it walks the real DOM and makes changes in place. Virtual DOM needs a full in-memory copy (memory = tree × 2). For LifeTrack's large AE tables, Ivy means rendering doesn't hold 200 virtual rows just to diff them.

Q34 Ways to control AOT compilation? general

CLI flags (`ng build --aot=true`), angular.json configurations, tsconfig (`strictTemplates: true`). Since Angular 9 (Ivy), AOT is the default in production AND development.

Q35 What is Angular Universal? general

Server-side rendering (SSR). Angular runs on Node.js and sends fully rendered HTML, which hydrates into an interactive app. Benefits: SEO, faster First Contentful Paint. LifeTrack (login-protected internal tool) probably doesn't need it — SEO doesn't matter for authenticated apps.

Q36 Do I need a Routing Module always? in project

No — a separate Routing Module is a convention, not a requirement. Your project uses AppRoutingModule and DashboardRoutingModule for organisation. Small apps can define routes inline. Standalone Angular uses `provideRouter(routes)` — no routing module needed.

Q37 Purpose of a wildcard route? in project

`path: '**'` matches any URL not matching other routes — placed last. Your project: `{ path: '**', redirectTo: 'dashboard' }`. Any unknown URL redirects to /dashboard, then AuthGuard runs. Common use: 404 page.

Q38 What is ActivatedRoute? in project

An injected service giving access to the active route: URL params, query params, static data, snapshot. Your RoleGuard reads `route.data?.['roles']`. A detail page reads `this.route.snapshot.paramMap.get('id')` (once) or subscribes to `paramMap` (live).

Q39 What is router state? in project

A tree representing all active routes, their params, data, and components — via `router.routerState`. Your NavigationService captures changes via `NavigationEnd.urlAfterRedirects` to track the previous URL.

Q40 What is router-outlet? in project

A placeholder directive where Angular renders the matched child route's component. Your DashboardComponent has one — URL /dashboard/ctm renders CtmDashboardComponent there, /dashboard/adverse-events renders the AE page. Single primary outlet.

Q41 What are dynamic components? general

Components created and inserted at runtime via `ViewContainerRef.createComponent()` rather than declared in a template. LifeTrack uses *ngIf for modals (simpler). Dynamic components fit a global toast/notification system needing components outside the current template hierarchy.

Q42 How do custom elements work internally? general

`createCustomElement()` wraps an Angular component in a custom element class the browser registers. Internally maps @Input() → attributes, @Output() → DOM events, lifecycle hooks → custom element lifecycle. Useful for embedding LifeTrack widgets in a non-Angular portal.

Q43 Utility functions provided by RxJS? in project

Three categories: **Creation** (of() in your catchError fallbacks, EMPTY in AuthInterceptor, from(), interval()), **Combination** (forkJoin() in all dashboards, combineLatest, merge, zip), **Pipeable operators** (map, tap, catchError, filter, takeUntil, timeout, finalize, debounceTime, switchMap).

Q44 How do you perform error handling in observables? in project

Three levels in your project: (1) per-request `catchError(() => of({items:[]}))` in pipe; (2) subscribe-level error callback for user messages; (3) global AuthInterceptor catchError for 401 → logout(). Rule: global for auth, per-request for fallbacks, subscribe-level for user feedback.

Q45 What is multicasting? in project

One Observable execution shared among multiple subscribers (vs cold where each triggers a separate execution). Subject/BehaviorSubject are multicast. Your `currentUser$` is shared by DashboardComponent, CtmDashboard, AuthGuard, AuthInterceptor — one `next(user)` notifies all four.

Q46 What is subscribing? in project

Attaching an observer to an Observable, triggering its execution. Without .subscribe(), an Observable is inert — no HTTP fires. Your components subscribe in ngOnInit; long-lived subscriptions pair with `takeUntil(destroy$)` cleaned up in ngOnDestroy.

Q47 Error handling for HttpClient? in project

Three patterns: (1) silent resilient fallback (DashboardService timeout + catchError → of(0)); (2) user-visible message (submitReport error callback → reportError); (3) global 401 handler (AuthInterceptor → logout + EMPTY). The cascade: interceptor handles 401, per-request catchError handles timeouts/5xx, subscribe-level handles user feedback.

Q48 What is a parameterized pipe? in project

A pipe accepting arguments after a colon. Your project: `{{ ae.reportedDate | date:'mediumDate' }}`, `{{ n.createdDate | date:'MMM d, y · h:mm a' }}`, `{{ r.enrollmentRate | number:'1.1-1' }}`. The date pipe takes format tokens; the number pipe takes 'minInt.minFrac-maxFrac'.

Q49 How do you categorize data binding types? in project

Four types by direction: **Class → Template** (interpolation `{{ }}`, property `[disabled]`); **Template → Class** (event `(click)`); **Both** (two-way `[(ngModel)]`). Your AE page uses all four: `{{ ae.description }}`, `[class.input-err]="..."`, `(click)="openReviewModal(ae)"`, `[(ngModel)]="searchTerm"`.

Q50 What happens if you use a script tag inside a template? general

Angular strips `<script>` tags at compile time — silently removed, never executed. A deliberate XSS protection. Even `[innerHTML]` is sanitized. To inject trusted HTML, use `DomSanitizer.bypassSecurityTrustHtml()` — rare, used with extreme care.

Q51 Lifecycle hooks for components and directives? in project

Eight in order: constructor (DI), ngOnChanges (@Input changes), ngOnInit (HTTP calls — your workhorse), ngDoCheck, ngAfterContentInit, ngAfterViewInit (chart canvas init), ngAfterViewChecked, ngOnDestroy (your destroy$.next()/complete() in CtmDashboard, NotificationsPage).

Q52 When store Subscription and unsubscribe() in ngOnDestroy vs ignore? in project

Always unsubscribe: long-lived subscriptions (HTTP in ngOnInit, router events, intervals, BehaviorSubject). **Can ignore:** single-emission HTTP (Angular cleans up after completion), AsyncPipe (`| async` auto-managed). Your clean pattern: `takeUntil(destroy$)` + ngOnDestroy — one destroy$.next() kills everything.

Q53 How to set headers for every request? in project

An HttpInterceptor. Your AuthInterceptor: `req.clone({ setHeaders: { Authorization: \`Bearer \${token}\` } })`. Registered with `{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }`. multi:true is critical — without it, it replaces the interceptor array.

Q54 How to detect a route change? in project

Subscribe to `router.events` and filter for the event type. Your NavigationService: `this.router.events.pipe(filter(e => e instanceof NavigationEnd)).subscribe(...)`. NavigationEnd fires after navigation commits. Use cases: analytics, breadcrumbs, scroll reset, page titles.

Q55 Need for SystemJS in Angular? general

SystemJS was a runtime module loader used in early Angular 2 (2016–17) before bundlers. Essentially obsolete. Modern Angular CLI uses esbuild (17+) or Webpack — the browser receives pre-bundled JS. Your LifeTrack project uses ng serve/ng build with esbuild; SystemJS is irrelevant.

NOTE ON Q56–Q103

This chat answered Q1–Q55 in two batches before the summary was requested. Questions Q56–Q103 (covering localStorage vs cookies, change detection, NgZone, Ivy/Incremental DOM internals, lazy loading mechanics, RxJS switchMap/mergeMap/concatMap, cold vs hot Observables, debouncing/throttling, TypeScript config, view encapsulation, and more) were not yet answered in the session and are therefore not included in this document.

CLOSING

Overall Learning Summary

This document captures a complete Angular tutoring session grounded entirely in the LifeTrack clinical-trial-management project. Nine topics were taught, each with definitions, sub-topics, real project code, and general examples; 55 interview questions were answered.

Session at a glance

Metric	Count
Core topics taught	9 (of an 11-topic curriculum)
Interview Q&As answered	55 (Q1–Q55)
Lifecycle hooks covered	8
RxJS operators covered	10 (7 used in project)
Guard types covered	4 (CanActivate, CanDeactivate, Resolve, CanActivateChild)
Data binding types	4
Deep-dive responses	2 (Router Features, AuthInterceptor flow)

The nine topics and their core insights

1. Introduction & Setup

Concepts: Angular CLI, ng new/serve/build, angular.json, main.ts, bootstrapApplication, environment.ts. Core insight: `ng serve` compiles in memory through 7 phases (config → tsc → ngtsc AOT → esbuild → Vite → browser → HMR watch). Nothing writes to /dist/ until `ng build`.

2. Components

Concepts: @Component decorator, four binding types, eight lifecycle hooks, @Input/@Output. Core insight: components are UI bricks; data flows DOWN via @Input(), events flow UP via @Output(); the template mirrors the TypeScript class.

3. Directives & Pipes

Concepts: *ngIf/*ngFor/*ngSwitch, ngClass/ngStyle/ngModel, custom directive, date/number pipes, custom pipe. Core insight: *ngIf removes from the DOM entirely (not display:none); *ngFor creates one element per item; pipes transform for display only — original data is never mutated.

4. Forms

Concepts: template-driven (AE Report modal) and reactive (Login/Register/Change Password), FormBuilder/FormGroup/FormControl/FormArray, built-in + custom + cross-field validators. Core insight: template-driven = Angular reads your HTML and builds the model; reactive = you build the model in TypeScript first. Use reactive for complex/cross-field validation.

5. Dependency Injection & Services

Concepts: @Injectable, providedIn:'root', hierarchical DI, AuthService/DashboardService/NavigationService, guards, AuthInterceptor. Core insight: providedIn:'root' means one singleton shared everywhere — when AuthService.currentUser$ emits, every subscriber reacts simultaneously.

6. Routing & Navigation

Concepts: forRoot/forChild, router-outlet, lazy loading, nested routes, route params, query params, static data, RoleHomeComponent, the 28-route tree. Core insight: lazy loading downloads a module bundle only when first navigated to; the router-outlet is where child components render; route data carries static config (roles, title) that guards and components read.

7. HTTP Client & APIs

Concepts: GET/POST/PUT/DELETE, forkJoin, catchError/map/tap/timeout/takeUntil/finalize, EMPTY/throwError/of, AuthInterceptor. Core insight: the interceptor is invisible middleware — every request passes through automatically. The static logoutInFlight flag prevents 7 parallel 401s from calling logout() 7 times.

8. State Management

Concepts: local/shared-service/server state, BehaviorSubject vs Subject, NgRx concepts. Core insight: BehaviorSubject is a box that holds a value AND has a loudspeaker — late subscribers still get the current value; .asObservable() hides .next() so only the service can push.

9. Reactive Programming (RxJS)

Concepts: Observable/Observer/Subject, creation/combination/pipeable operators, the four reactive patterns. Core insight: think of data as a conveyor belt — each operator is a worker, subscribe() is the person at the end; the belt doesn't move until someone subscribes.

The three patterns most worth talking about in an interview

> **1. THE AUTHINTERCEPTOR**
>
> The static `logoutInFlight` flag, the `req.clone()` immutability, the `EMPTY` vs `throwError` distinction, and the `multi:true` registration form a senior-level design decision you can explain line by line.

> **2. THE BEHAVIORSUBJECT PATTERN IN AUTHSERVICE**
>
> One service, one subject, every consumer gets the same user simultaneously — and late subscribers still get the value because it replays. This answers most state-management questions.

> **3. FORKJOIN + PER-REQUEST CATCHERROR + TAKEUNTIL(DESTROY$)**
>
> Your dashboard loads 7 things in parallel, any one can fail safely, and everything cleans up when the component is destroyed. The pattern that separates knowing the API from knowing Angular.

Pending from the curriculum

Topics 10 (Testing Angular Applications) and 11 were not reached in this session. Interview questions Q56–Q103 were also not yet answered. The LifeTrack project has `.spec.ts` files in place; the patterns to test (services with spies, components with TestBed, HTTP with HttpClientTestingModule) are all real and waiting for a future session.